Zurich ServiceNow AI Platform Capabilities

Last updated: January 16, 2026

Some examples and graphics depicted herein are provided for illustration only. No real association or connection to ServiceNow products or services is intended or should be inferred.

This PDF was created from content on docs.servicenow.com. The web site is updated frequently. For the most current ServiceNow product documentation, go to docs.servicenow.com.

Company Headquarters
2225 Lawson Lane
Santa Clara, CA 95054
United States
(408)501-8550

CMDB Identification and Reconciliation (IRE)

The Identification and Reconciliation module provides a centralized framework for identifying and reconciling data from different data sources. It helps maintain the integrity of the CMDB and some non-CMDB tables when multiple data sources are used to create and update CI records.

Get started

Explore  Learn about benefits and how IRE helps maintain the integrity of the CMDB.

Configure  Configure identification, reconciliation, and other rules.

IRE and Import Sets  Apply IRE to Import Sets that are importing CIs into the CMDB.

Identification rules  Create the rules that uniquely identify CIs based on CI attributes.

Reconciliation rules  Create the rules that manage authorization and update priority between discovery sources.

IRE and Duplicate CIs  Learn and configure how IRE detects duplicate CIs and how to process de-duplication tasks.

IRE and non-CMDB tables  Apply

Identification Simulator  Construct a

Reference  Properties

IRE processes to supported non-CMDB tables.	validated payload and simulate running it through IRE processes.	and tables associated with IRE, and how domain separation is supported in IRE functions.

Troubleshoot and get help

- Ask questions and explore other resources for in the ServiceNow Community
- Search the Known Error Portal for known error articles
- Contact Customer Service and Support
- Exploring CMDB Identification and Reconciliation (IRE)

  The Identification and Reconciliation (IRE) module provides a centralized framework for identifying and reconciling data from different data sources. It helps maintain the integrity of the CMDB and some non-CMDB tables when multiple data sources are used to create and update CI records.

- Configuring CMDB Identification and Reconciliation

  Configure the necessary rules for CMDB Identification and Reconciliation (IRE) to function effectively.

- Applying IRE to Import Sets

  You can apply CMDB Identification and Reconciliation Engine (IRE) processes when Import Sets are used to import CIs into the CMDB. CI identification can prevent duplicate CIs in the CMDB, which Import Sets might otherwise cause.

- Detecting duplicate CIs

  When IRE identification process detects duplicate CIs, it groups each set of duplicate CIs into a de-duplication task for review and remediation. A large number of duplicate CIs might be due to weak identification rules. You can configure the identification engine to reconcile duplicate CIs.

- Using identification simulation

  Identification simulation is a central location for automatically constructing a payload that is guaranteed to be complete and valid. You can then simulate the processing of the payload by the Identification and Reconciliation Engine (IRE) and examine the results before actually submitting it for execution by IRE.

- View a reclassification task

  Reclassification tasks are created for CIs that couldn't be automatically reclassified during the identification process. Review these tasks to locate the CIs and decide if to reclassify them.

- IRE support for non-CMDB tables

  Apply Identification and Reconciliation Engine (IRE) processes to supported non-CMDB tables to ensure data integrity and health of those tables.

- CMDB IRE reference

  Reference topics provide property settings, domain separation, and other reference content for the CMDB Identification and Reconciliation Engine (IRE).

Exploring CMDB Identification and Reconciliation (IRE)

The Identification and Reconciliation (IRE) module provides a centralized framework for identifying and reconciling data from different data sources. It helps maintain the integrity of the CMDB and some non-CMDB tables when multiple data sources are used to create and update CI records.

The use of multiple data sources increases the risk of introducing inconsistencies through duplicate records. To maintain the integrity of the database, it is important to correctly identify CIs and services so that new records are created only for CIs that are truly new.

The Identification and Reconciliation Engine (IRE) helps maintain the data integrity as follows:

- Prevent duplicate CIs by uniquely identifying CIs using sets of identification rules

- Reconcile CI attributes by allowing only authoritative data sources to write to the CMDB or to a supported non-CMDB table
- Reclassify CIs
- Provide a centralized framework to perform identification and reconciliation across different data sources

Data sources such as ServiceNow Event Management, Horizontal Discovery, Import Sets, Cloud Insights, Pattern Discovery, and Manual Entry, use IRE APIs to perform CI identification and reconciliation. In addition, any 3rd party data source can leverage REST/Scriptable IRE APIs to also perform CI identification and reconciliation.

Support for non-CMDB tables

IRE processes support some non-CMDB tables. You can create identification rules, reconciliation rules, and other IRE-related rules to ensure the integrity of data inserted or updated in supported non-CMDB tables. For details, see IRE support for non-CMDB tables.

- Components and process

  The CMDB Identification and Reconciliation functionality is supported by the Identification and Reconciliation engine (IRE), rules, and tasks. Identification rules, reconciliation rules, IRE data source rules, de-duplication tasks, and reclassification tasks determine how IRE identifies and reconciles CI.

Components and process

The CMDB Identification and Reconciliation functionality is supported by the Identification and Reconciliation engine (IRE), rules, and tasks. Identification rules, reconciliation rules, IRE data source rules, de-duplication tasks, and reclassification tasks determine how IRE identifies and reconciles CI.

Concepts and Components of Identification and Reconciliation

Identification

Process of uniquely identifying CIs, to determine if a CI already exists in the CMDB or if it is a newly discovered CI that must be added to the

CMDB. Identification processes rely on identification rules, or on unique IDs for CIs that data sources can provide.

Reconciliation

Process of reconciling CIs and CI attributes by allowing only designated authoritative data sources to write to the CMDB at the CI table and attribute level. The CMDB is updated in real time as records are being processed. There is no staging area to verify the reconciliation activities before they are committed. Reconciliation processes rely on reconciliation rules and IRE data source rules.

Reconciliation is required only for update operations, when the identification process identifies a CI in the CMDB that matches an incoming CI in the payload. When IRE inserts a new CI, reconciliation is not applied.

De-duplication tasks

If the instance encounters duplicate CIs during the Identification and Reconciliation process, it groups each set of duplicate CIs into a de-duplication task. Review the information in these tasks to see how it was determined that these CIs are duplicates.

Reclassification tasks

During the CI identification process, a matched CI might need to be upgraded, downgraded, or switched to another CI class. If automatic reclassification is disabled, then the system generates a reclassification task. Review the information in these tasks, and decide whether a manual reclassification of the CI is appropriate.

APIs

The Identification and Reconciliation APIs are a centralized set of APIs that can be used with different sources of data such as Discovery, Monitoring, or Import Sets. You can use it to enforce Identification and Reconciliation before data is stored in the CMDB. Data sources do not directly write to the CMDB. Instead, they call APIs first to ensure that the data being written does not introduce inconsistencies.

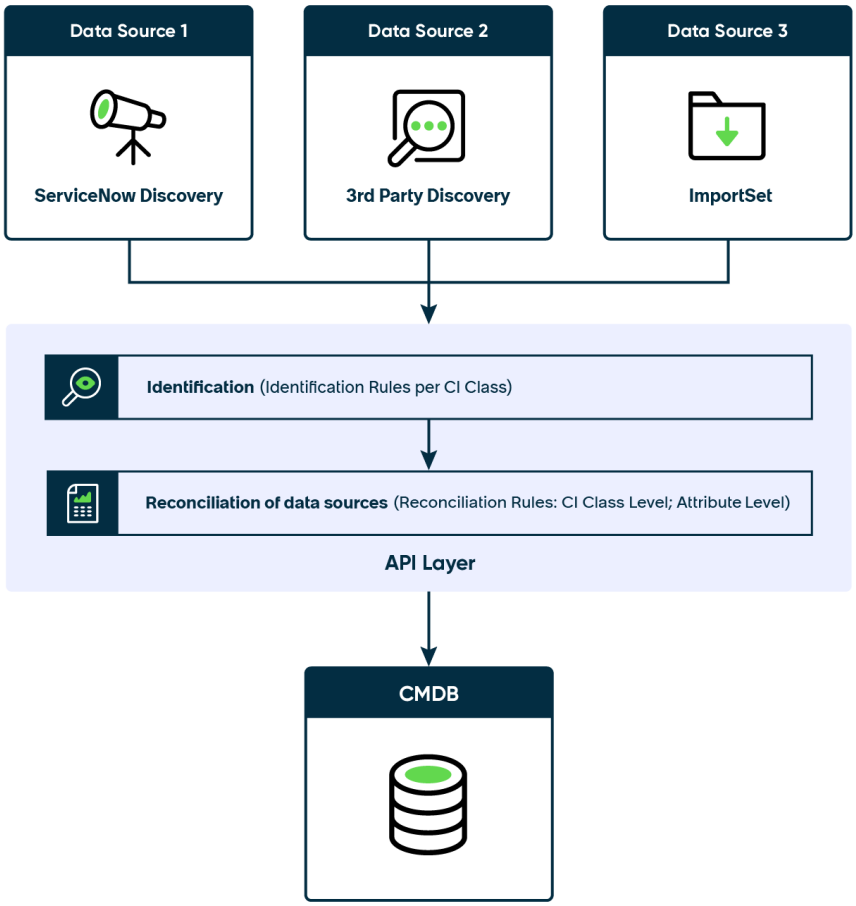
Identification engine APIs are accessible in scoped apps. The Configuration Management For Scoped Apps (CMDB) plugin (com.snc.cmdb.scoped) allows a scoped app in scripts to use the prefix 'sn_cmdb.IdentificationEngine.<method>' to access identification engine

APIs. The Configuration Management For Scoped Apps (CMDB) plugin is activated in base systems.

- createOrUpdateCI(): A scriptable API that creates or updates a CI based on identification and reconciliation rules.
- identifyCI(): Similar to the createOrUpdateCI API, but does not commit the result to the database. Use this API with a given payload to find out if the identification engine will perform insert or update operations, without committing the operation.
- CreateOrUpdateCIEnhanced(): A scriptable API that provides the functionality of enhanced IRE features such as partial payload, partial commit, incomplete payload, and deduplication of payload items. You can select the enhanced features to use. However, if you enable partial payloads, then deduplication of payload items and partial commit are automatically enabled.
- identifyCIEnhanced: Similar to the createOrUpdateCIEnahnced API, but does not commit the result to the database. Use this API with a given payload to find out if the identification engine will perform insert or update operations, without committing the operation.
- CMDBTransformUtil: An API to be used exclusively with Import Sets to apply Identification and Reconciliation processes to data imported by Import Sets.

Predefined identification are included for many of the tables in the base system. You can customize these rules for your organization. When a new table is created in the CMDB, it derives identification and reconciliation rules from its parent table if these rules exist. To apply identification and reconciliation rules to a new table, create the rules either at the child level or at its parent level.

Process flow of Identification and Reconciliation



Identification and Reconciliation Engine (IRE)

Identification and Reconciliation engine (IRE) is a rule-based engine, operating as an underlying key component in Identification and Reconciliation. IRE provides a centralized framework to perform identification and reconciliation processes across different data sources. IRE uses identification rules, reconciliation rules, and IRE data source rules when processing incoming data before inserting or updating data in the CMDB. IRE processes help maintain data integrity in the CMDB.

- IRE prevents duplicate CIs by uniquely identifying CIs.
- IRE reconciles CI attributes by allowing only authoritative data sources to write to CMDB.

Identification and Reconciliation engine (IRE)

IRE is an underlying key component in Identification and Reconciliation, providing a centralized framework to perform identification and reconciliation processes across different data sources. IRE uses identification rules, reconciliation rules, and IRE data source rules when processing incoming data before inserting that data to the CMDB.

IRE processes help maintain data integrity in the CMDB.

- IRE prevents duplicate CIs by uniquely identifying CIs.
- IRE reconciles CI attributes by allowing only authoritative data sources to write to CMDB.

ServiceNow® applications such as Service Mapping, horizontal discovery, and pattern discovery, use APIs to apply identification and reconciliation processes. You can also apply IRE processes to data imported by import sets. When using other data sources including third-party data sources, you can leverage REST or scriptable IRE APIs to perform identification and reconciliation.
Additional information:

- About properties that affect some functions of IRE: See Properties.
- About using IRE APIs: See IdentificationEngine - Scoped.
- About enabling debugging and checking on issues with a payload: See How to log the payload sent to IRE and check the issues with the payload [KB0750382].
- About the steps that IRE performs illustrating how IRE works, such as validating the payload, applying reconciliation, and committing the data to the CMDB: See [CMDB - IRE] How the CMDB Identification and Reconciliation Engine works when passing a CI (as payload) to the createOrUpdateCI() [KB0750386].
- About how to run a payload through IRE: See [CMDB IRE] How to run the CI identification on demand using the payload [KB0750383].

CI identification

The CMDB identification process relies on identification rules to uniquely identify CIs. When possible, CIs can also be uniquely identified using source_name and source_native_key values provided in the sys_object_source_info section of the payload, and the Source [sys_object_source] table. If identification is successful using that method, then it is not necessary to apply matching algorithms that rely on identification rules, which is a slower identification method.

You can configure the system to prioritize the use of IRE identification rules, even when source_name and source_native_key are provided and can be used for unique identification. For more information about using the glide.identification_engine.skip_sys_object_source_matching system property to manage this behavior, see Properties.

A unique CI identifier can be provided in the optional sys_object_source_info object in the IRE payload.

```
{
  "items": [
    {
      "className": "cmdb_ci_win_server",
      "values": {
        "name": "SAMLABVM52"
      },
      "sys_object_source_info": {
            "source_native_key": "16777219",
            "source_name": "SCCM",
            "source_feed": "SCCM Computer Identity",
            "source_recency_timestamp": "2019-08-26 13:00
:00"
      }
     }
    ]
  }
```

Identification processes rely on CIs dependency classification to uniquely identify CIs. For example, to identify a Tomcat CI which is a dependent CI. Assuming a Windows Server CI (independent class) which is running a Tomcat application (dependent class). elying on 'config file path' to uniquely identify the Tomcat CI, isn't sufficient because the Tomcat application can run on multiple machines with identical paths. The identification engine won't be able to choose a CI to update.

Dependent relationships force the identification process to first identify the Windows Server host on which the Tomcat application is running, and only then, in the context of the host, to uniquely identify the Tomcat application itself.

Payload items identification

IRE generates identifier keys for all payload items in an incoming payload and then uses those keys when trying to match partial and incoming payloads. An identifier key is based either on:

- Combination of the source_name and source_native_key values from the sys_object_source_info object.
- Identification criterion attributes.

IRE stores the identifier keys associated with partial items in the CMDB IRE Partial Payloads Index [cmdb_ire_partial_payloads_index] table, and then uses those keys to try to match with identifier keys of incoming payloads.

Timestamps in key attributes

To help resolve conflicting attribute values, IRE uses timestamps in the following attributes to identify records that are older than the current record and therefore can be ignored:

-

  Most recent discovery (last_discovered) and Discovery source (discovery_source):

  Most recent discovery (last_discovered) is the timestamp of when the CI was last discovered. IRE always updates CIs' last_discovered and discovery_source attributes during payload processing, even when no other CI attributes are updated. When last_discovered is provided in the payload, IRE updates the CI with the provided value only if the last_discovered time in the payload is newer than the one in the CMDB. If last_discovered is not provided in the payload, IRE updates the last_discovered attribute with the current timestamp.

  You can use the glide.identification_engine.skip_updating_source_last_discovered_if_older and the

glide.identification_engine.ire_message_listener_skip_updating_source_last_discovered_to_now system properties to modify this default behaviour.

- First discovered (first_discovered) is the timestamp of when the CI was first created.
  - When the CI is first created: If a value is provided in the payload, IRE inserts that value. Otherwise, IRE inserts the current timestamp.
  - In subsequent updates: If a value is provided, IRE updates the CI with the provided value. Otherwise, the attribute is not updated.

You can also use the following system properties to modify how IRE uses the source_recency_timestamp value in a payload to update the last_scan attribute in the Source [sys_object_source] table:

- glide.identification_engine.skip_updating_last_scan_if_older
- glide.identification_engine.ire_message_listener_skip_updating_last_scan_to_now

Enhanced IRE features

The CreateOrUpdateCIEnhanced() and identifyCIEnhanced scriptable APIs provides the functionality for the following enhanced IRE features, which can be enabled or disabled as needed:

Partial payloads

IRE isolates items for which data sources did not provide enough information to uniquely identify the CI and therefore processing cannot continue. Some of these items are identified as partial items, which get stored for potential later processing. Other items are identified as incomplete items, which get stored for logging purposes only.

For example: SCCM has multiple feeds such as a disk feed and a computer feed. The disk feed might have complete information about the disk but insufficient information about the computer CI that it depends on.

API option: partial_payloads which is enabled by default. When partial_payloads is enabled, partial_commits and deduplicate_payloads are automatically enabled regardless of their setting in options.

Partial commits

Errors in some items do not prevent committing the rest of the items in a payload. Therefore, when a payload contains items with errors, IRE still commits the remaining valid items in the payload. In this situation, some of the uncommitted items are saved as partial payloads and other uncommitted items are saved as incomplete payloads.

API option: partial_commits which is enabled by default.

Deduplicate payload items

IRE deduplicates duplicate items within the payload, merging those duplicates into a single payload item for processing.

API option: deduplicate_payloads which is enabled by default.

Generate summary

IRE generates summaries in the output payload with processing details such as the number of updates per class.

API option: generate_summary which is disabled by default.

Partial items

A payload item is determined to be a partial item if it doesn't contain the necessary data for unique identification and if it has one of the following errors. Unique identification requires that the payload item has the sys_object_source_info section with source_name and source_native_key values, or the full set of the identification criterion attributes specified for the CI class, or both.

IRE errors for a partial item:

- MISSING_MATCHING_ATTRIBUTES —— Item does not have identification criterion attributes to use at least one identifier entry for matching.
- REQUIRED_ATTRIBUTE_EMPTY —— Unable to create a CI because a required attribute is missing.
- MISSING_DEPENDENCY —— Dependent CI is missing a dependency relation which is specified in the payload.

- INSERT_NOT_ALLOWED_FOR_SOURCE —— An IRE data source rule prevents the specified data sources from creating CIs of the specified class.

For more details about IRE error messages, see IRE error messages.

If processing fails because payload items are determined to be partial items, then the partial items are saved as partial payloads in the CMDB IRE Partial Payloads [cmdb_ire_partial_payloads] table in JSON format for later potential processing. IRE uses identifier keys to attempt to match incoming payloads with stored partial payloads.

If later, a data source sends the data that was missing in the partial item, IRE matches the incoming payload with the stored partial payloads. IRE then merges any matching partial payloads with the incoming payload. To resolve any conflicting attributes, IRE uses either source_recency_timestamp (when source_native_key and source_name are identical), or static reconciliation rules specified for the class. The result is a complete and valid payload which IRE then processes to create or update the respective CIs.

Partial payloads older than 90 days are deleted from the CMDB IRE Partial Payloads [cmdb_ire_partial_payloads] table.

Sample of a partial payload:

```
Disk feed:
{
  "items": [
    {
      "className": "cmdb_ci_computer",
      "sys_object_source_info": {
            "source_native_key": "Server001",
            "source_name": "SCCM",
            "source_feed": "DISK_FEED",
            "source_recency_timestamp": "2019-08-26 13:00
:00"
      }
    },
    {
      "className": "cmdb_ci_disk",
      "values": {
        "name": "disk1"
      }
```

```
    }
  ],
  "relations": [{
              "parent": 0,
              "child": 1,
              "type": "Contains::Contained by"}
              ]
}
```

The computer item in the above payload does not have any attributes and therefore IRE can't process it. However, source_name and `source_native_key` are provided making it a partial item. Because the computer item is partial, the disk item that depends on the computer item, is a partial item too.
Sample of a subsequent payload that completes the previous partial payload by providing the missing details:

```
Server/Computer feed:
 {
  "items": [
    {
      "className": "cmdb_ci_linux_server",
      "values": { "name": 'linux001',
                "ip_address": "100.126.38.19",
                "mac_address": "DSWER4587" },
      "sys_object_source_info": {
            "source_native_key": "Server001",
            "source_name": "SCCM",
            "source_feed": "COMPUTER_IDENTITY",
            "source_recency_timestamp": "2019-08-26 14:00
:00"
      }
    }
  ]
}
```

The computer in the partial payload and the server in the new payload match because they have identical source_name and source_native_key. Therefore, the partial payload and the new payload are merged, the operation is committed, and the partial payload is deleted from the Partial Payloads table.

There is a limit on the number of items per partial payload, which is set by the glide.identification_engine.partial_payload_items_max_size property (1000 by default). Storing associated relationships, references,

and dependent items, in one partial payload, can result in reaching that limit, in which case, the payload is split into multiple partial payloads.

For more information about partial payloads, see CreateOrUpdateCIEnhanced().

Incomplete items

A payload item is determined to be an incomplete item if:

- It does not contain all the data necessary for unique identification
- It has an error that is not associated with a partial item

Unique identification is not possible if neither source_name and source_native_key within the sys_object_source_info object, nor the full set of identification criterion attributes specified for the CI class, is provided.

Incomplete items are saved as incomplete payloads in the CMDB IRE Incomplete Payloads [cmdb_ire_incomplete_payloads] table in JSON format. Incomplete items are stored for the purpose of logging payloads with irrecoverable errors, and are never processed again.

Adding relationships

Add relationships by using either indices, or the optional JSON internal_id element.

Use the relations object in the payload to add or update relationships by referring to internal_ids of items. Relationships can be created using main items and related items in the payload. For example:

- Relation (parent Index, child Index, Relation Type)
- Relation (parent Internal Id, child Internal Id, Relation Type)

For more information and for code samples, see CreateOrUpdateCIEnhanced().

Adding references between payload items

Add references between two payload items by using the optional JSON internal_id element, which uniquely identifies payload items.

Use the referenceItems block to add or update references. You can add references between any two items, including main items, lookup items, and related items, in a single payload.

For more information and for code samples, see CreateOrUpdateCIEnhanced().

CI reclassification

Use the updateWithoutUpgrade, updateWithoutDowngrade, and updateWithoutSwitch flags in the settings block in a payload, to prevent unintentional updates to CIs' class. These flags prevent upgrading, downgrading, or switching the class of a CI that multiple data sources unintentionally might attempt while updating the same CI. For more information and for code samples, see CreateOrUpdateCIEnhanced().

Reclassification flags have precedence over any other system settings for Configure CI reclassification during IRE processing.

Adding custom before and after scripts

Use the IntegrationHub ETL to add custom Java scripts for a data source of a CMDB integration application. Those scripts provide access to the IRE input and output payloads, while processing CMDB integrations.

Before scripts provide access to a batch of input payloads that will be sent to IRE. Using a custom before script lets you:

- Skip a payload in the batch by setting the status to `SKIPPED`. Optionally, provide a reason for skipping the payload which will appear as a comment on the respective import set row table.
- Modify the input payload.
- Write other custom logic inside the script that uses the IRE payload.

After scripts provide access to the IRE input and output payloads. Using a custom after script lets you:

- Easily compare the input and output payloads and identify the different operations that IRE performed on each CI.
- Access the sys_ids of the CIs that IRE created or updated.

- Write other custom logic inside the script that uses the IRE output payload.

Configuring CMDB Identification and Reconciliation

Configure the necessary rules for CMDB Identification and Reconciliation (IRE) to function effectively.

Configuration overview

- Create identification rules

  Identification rules are essential in correctly identifying CIs in the CMDB to prevent the creation of duplicate CIs. Effective identification rules help determine whether an incoming CI is added to the CMDB or used to update and existing CI.

- Create reconciliation rules

  Static and dynamic reconciliation rules determine which discovery source is authorized to update which CI attributes. Reconciliation rules are important in preventing discovery sources from overwriting each other's updates to the same attribute values.

- Create data source rules

  Data source rules prevent discovery sources from inserting new CIs for a specific class. Data source rules are useful in preventing untrusted discovery sources from creating new CIs while continuing to trust these discovery sources in updating existing CIs.

- Create dependent relationship rules

  Dependent relationship rules define the dependency structure of the CI types and the relationship types in service definitions, which help in CI identification and in the construction of business service maps.

- Identification rules

  The CMDB identification process relies on identification rules to uniquely identify CIs.

- Reconciliation rules

Reconciliation rules determine which discovery sources can update CI attributes.

- Configure CI reclassification during IRE processing

  During the Identification and Reconciliation Engine (IRE) CI identification process, a CI might need to be reclassified to a different sys_class_name type. By default, CIs are reclassified automatically. If automatic reclassification is disabled, then the CI is not reclassified and the system generates a reclassification task for your review.

- Create an IRE data source rule

  When using Identification and Reconciliation Engine (IRE), you can prevent a specific discovery (data) source from inserting new CIs for a specific class. Create IRE data source rules for discovery sources that you don't trust in creating CIs but continue to trust in updating those CIs that exist.

- CMDB dependent relationship rules

  Service definitions consist of CI types and relationship types. Dependent relationship rules define the dependency structure of the CI types and the relationship types in these service definitions, helping in CI identification and in the construction of business service maps.

Identification rules

The CMDB identification process relies on identification rules to uniquely identify CIs.

An identification rule applies to a CI class and consists of a single CI identifier and one or more identifier entries and related entries, each with a different priority. Each identifier entry defines a unique attribute set with a specific priority and each related entry defines rules for identifying related items. Create strong identification rules that are set with the highest priority for the strongest identifier entries and related entries.

The identification process and identification rules use the CI's attributes for identification:

Unique attributes

Designated sets of criterion attribute values of a CI that can be used to uniquely identify the CI. Unique attributes can be from the same table or from derived tables.

Required attributes

Designated attributes of a CI that cannot be empty.

Derivation across the CMDB hierarchy

If no identification rule is explicitly defined for a child class, then the child class derives its identification rule, including any associated identification entries and related entries, from its parent class. Later, an own identification rule can be explicitly defined for the child class. In that case, the identification rule that was initially derived from the parent class, including any associated identification entries and related entries, is no longer in effect at the child class. Also, you must explicitly add identification entries and related entries in the newly created identification rule at the child class.

For example: The Hardware class identification rule has a related entry for the Software Instance table. This identification rule, including its associated related entry for the Software Instance table, is derived by the Computer class. If you then create a new identification rule for the Computer class, it overwrites the identification rule that was derived from the Hardware class. Therefore, the Hardware class identification rule, including its associated related entry for the Software Instance table, is no longer in effect for the Computer class. If the same related entry is needed, you must explicitly add a related entry for the Software Instance table in the newly created identification rule for the Computer class.

Identification rule types

CI dependency is specified in the CI Class Manager by the dependent relationship rules for the CI's class:

Independent CIs

CIs, such as Server CIs, which exist on their own and are not dependent on any other CIs.

Dependent CIs

CIs which depend on a relationship to another CI and can't exist on their own in the absence of the dependent relationship. For example:

- Network Adapter CIs can't exist meaningfully without the Hardware CIs that contain them.
- Application CIs can't exist on their own without the Server CI they are hosted on.

The steps for identifying dependent CIs can be different from the steps for identifying independent CIs. This difference is reflected in the differences between dependent identification rules and independent identification rules:

Independent identification rule

A rule that identifies a CI based on the CI's own attributes, independently of other CIs or relationship.

Dependent identification rule

A rule in which identifying a CI requires identifying a dependent CI first. A CI can have dependency on one or more CIs, and a dependent CI can have only a single parent CI with dependency. The relationship types between the CI and its dependent CIs are also included in the identification process. To help with the identification process of dependent CIs, create dependent relationships that define the dependency chain within CI types.

The payload used for identification of a dependent CI, can include a relationship with a qualifier chain. For such relationship, if there is a matching parent/child pair, the system compares the qualifier chain in the payload, with the qualifier chain of the CIs in the database. If there is a difference, the qualifier chain in the database is updated to match the qualifier chain in the payload for that relationship.

Identifier entries

You can configure an identifier entry to match a CI not only based on the CI's own attributes (field-based identification) but also based on the CI's related list (lookup-based identification) such as **Serial Numbers** or **Network Adapters**. The lookup table that is used for identification, needs to have a reference field that points to cmdb_ci.

There are three types of identifier entries:

Regular identifier entry

Identification is based on CI's own attributes that uniquely identify the CI.

Lookup identifier entry

Identification uses a lookup table (related table) which can be any table that has a reference to the CI that is being identified. After you select a related lookup table, you select identifier attributes from the related table that reference either the cmdb_ci table itself, or one of its descendants.

If the lookup records don't already exist, then they are inserted in the lookup table referenced in the identifier entry.

Hybrid identifier entry

A combination of both, a regular identifier entry and a lookup identifier entry.

Example: When discovering virtual machines in a cloud environment which might contain two virtual machines with an identical set of serial numbers. A lookup identifier entry for the Hardware table such as `[Table: Serial Number, Criterion Attributes: Serial Number, Serial Number Type]` cannot uniquely identify these two virtual machines. However, a hybrid identifier entry such as `[Table: Serial Number, Criterion Attributes: Serial Number, Serial Number Type + (Name field from main Hardware table)]` can uniquely identify the two virtual machines.

Guidelines for lookup tables

Follow these guidelines when specifying a lookup table in an identifier entry.

1. Ensure that lookup tables reference the cmdb_ci table.
2. It is preferable to enforce exact count match (check box **Enforce exact count match (Lookup)**) for a stronger identification rule. During lookup identification, this option enforces matching only on exact lookup records count match. See Create a CI identification rule for more details.

3. Do not create conflicting identification rules especially for lookup-based rule.
   Example: In a CI Identifier for the Hardware class, you specify a lookup-based rule for the Network Adapter class and you also define a CI Identifier for the Network Adapter class. Duplicates might potentially be created in the Network Adapter table, because there are contradicting rules to identify a unique CI in that table:
   - One rule that looks only at criterion attributes (CI identifier rule)
   - Another rule that looks at criterion attributes and referenced sys_id (lookup rule).

Example: CI with related items that needs to be inserted - sysId is available.

```
var payload = {
    items: [{
        className:'cmdb_ci_linux_server',
       related: [{
              className:'cmdb_ci_spkg',
              values: {
                name:'package1',
                version:'version1'
                }
        }],
        values: {
            sys_id:'194876usytrr65378098'
        }
}]
};
```

Related entries

You can define related entries which are rules that are based on related CIs. A related entry is based on a related table which can be any table (CMDB or non-CMDB) that has a reference to the CI that is being identified. Related entries let you create or update records on other tables in which the data is associated with the CI being identified by the identifier entries. Related entries are not used to directly identify CIs.

After you select a related table for the rule, the list in **Referenced field** is populated with fields from the related table that reference either the cmdb_ci table itself, or one of its descendants.

A related entry for a class is derived by child classes for which no related entries are specified.

- Create a CI identification rule

  Identification rules are used to uniquely identify CIs in the CMDB, as part of Identification and Reconciliation (IRE) processes. Each CMDB class can be associated with a single identification rule.

- Create an identification inclusion rule

  Narrow the scope of CIs that are included in the identification process by creating an identification inclusion rule.

- General guidelines for using CMDB Identification

  Review the following general guidelines for using CMDB Identification effectively.

Related concepts

- Relation qualifier

Create a CI identification rule

Identification rules are used to uniquely identify CIs in the CMDB, as part of Identification and Reconciliation (IRE) processes. Each CMDB class can be associated with a single identification rule.

Before you begin

You can update a CI identification rule only at the class level that the rule is defined for. You can't update a derived rule.

Role required: itil has read access, itil_admin (on top of itil) has full access.

About this task

In a CI identification rule, specify a CI identifier, and identifier entries and related entries that uniquely identify the CI.

Review the following before creating identification rules:

- Identification rules
- General guidelines for using CMDB Identification
- Explore predefined identification rules:
  1. Navigate to **All > CI Class Manager**.
  2. Select **Hierarchy** and then search and select, for example, the Hardware class.
  3. In the Hardware bar, expand **Class Info** and select **Identification Rule**.
  4. Examine all the sections and tiles with the settings of the Hardware class identification rule.

Procedure

1. Navigate to **All > Configuration > CI Class Manager**.
2. Select **Hierarchy** to show the CI Classes list and then select the class for which to create an identification rule.
3. In the class navigation bar, expand **Class Info** and then select **Identification Rule**.
4. Select **Edit**, **Add**, or **Replace** (for a class that has derived the CI identification rule), in the Identification Rule section to create one.
5. Fill out the form, and then select **Save**.

Field	Description
Independent/Dependent	Designation of whether the CI identifier can identify the CI independently of other CIs, or not. **Note:** To set the rule as **Dependent**, you must specify dependent relationship rules for the selected class.

Field	Description
Name	Name of CI identifier.
Description	Description of the CI identifier.

6. In the Identifier Entries section, select an existing identifier entry to edit, or select **Add** to create one.
7. In the Identifier Entry dialog box, choose an option and then select **Next**.
   Continue with one of the following three steps according to the option you selected:

Option	Description
Use attributes from main table <table>	Lets you select attributes from the currently selected table (regular identifier entry).
Use attributes from another table (Lookup table)	Lets you select attributes from any related table, other than the currently selected table (lookup identifier entry).
Use attributes from main and another table (Hybrid)	Lets you select attributes from both the currently selected table, and from another table (hybrid identifier entry).

8. **Use attributes from main table <'table'>** option: Set the options on the form and then select **Save**.

   Search On Table is preset to the currently selected table in the CI Classes list.

Field	Description
Active	Check box that specifies the identifier entry is active. At least one identifier entry in an

Field	Description
	identification rule must be active for the rule to apply.
Priority	Priority of the identifier entry. Identifier entries are applied based on priority. Rules with lower priority numbers are given higher priority. Identifier entries of identical priorities are applied randomly. You can keep gaps between the priority numbers, so you can assign the unused priority numbers to new entries without modifying the existing priority order.
Criterion Attributes	Set of attributes that uniquely identify the CI. Attributes can belong to the current class, or to a parent class.

Field	Description
	Note: It is possible to add reference fields as a criterion attribute. However, such fields might not always be effective: • Reference fields store sys_ids that point to a record in another table, and thus is considered a weak criterion attribute (in terms of uniqueness) for the current table. • The system detects and then replaces invalid values in a reference field with 'Unknown'. For example, an invalid Model ID value is replaced with the value 'Unknown'. Also, if several CIs end up having that same reference field set to 'Unknown', then these CIs become duplicate CIs.
Allow null attribute	When selected, then if at least one criterion attribute is not null, attempt matching with an identifier entry even if there are criterion attributes that are null. Otherwise, all criterion attributes must have values to attempt matching with an identifier entry.

Field	Description
Allow fallback to parent's rules	Allows the identification rules of the CI's parent to be used if a match is not found for this identification rule. Applies only for dependent identification rules.
Advanced Options	A filter to narrow the set of records that will be searched for a matching CI. Available only if the glide.identification_engine.enable_identifier_optional_condition system property is set to **true** (**false** by default). In the base system, identifier entries of various classes are pre-configured with advanced options conditions. All these pre-configured conditions in regular identifier entries will automatically apply when you set this property to **true**. Therefore, to prevent unexpected behavior, review those predefined conditions in regular identifier entries before setting this property to **true**. For more details about this property, see Properties.

Note: If criterion attributes have only two attributes and sys_class_name is one of them (for example [name, sys_class_name], [ip_address, sys_class_name]), then the other attribute cannot be NULL, even if **Allow null attribute** is enabled. This restriction is due to sys_class_name being considered a special system matching attribute.

9. **Use attributes from another table (Lookup table)** option:

   a. Set **Search On Table** to a table other than the currently selected table in the CI Classes list.
   The **Search On Table** must have a reference field to cmdb_ci, otherwise the identifier entry is considered invalid.

   b. Set the rest of the fields as described in the previous step.

   c. (Optional) Select **Advanced options** and enter the information for a lookup identifier (scroll down if necessary).

Advanced Option	Description
All of these conditions must be met	A filter to narrow the set of records that will be searched for a matching CI.
Enforce exact count match	For lookup identification, match a CI only on exact lookup records count match. When enforced, all lookup items for a CI in the payload must have matching records in the lookup table, that reference the same CI: a. Only matches CIs that have all the lookup items from the input payload referencing the CI in CMDB. b. If there are multiple matches, selects the oldest created CI as the final match. When not enforced, one lookup item for a CI in the payload matching a record in the lookup table, is sufficient to consider a match:

Advanced Option	Description
	a. Matches any CI that has at least one of the lookup items from the input payload referencing the CI in CMDB. b. If there are multiple matches, selects the CIs with the max number of lookup items from the input payload referencing the CI in CMDB. c. If there are still multiple matches, selects the oldest created CI as the final match.

d. Select **Save**.

10. **Use attributes from main and another table (Hybrid)** option:

    a. Set the options on the **General Settings** tab as described in previous steps, and then select **Next**.

    b. On the **Main Table Settings** tab, select the attributes to use from the currently selected table, and then select **Next**.
    Search On Table is preset to the currently selected table in the CI Classes list.

    c. On the **Lookup Table Settings** tab, select a **Search On Table** and then in **Criterion Attributes** select attributes from the specified table. **Search On Table** must have a reference field to cmdb_ci, otherwise the identifier entry is considered invalid.
    You can select **Advanced options** and enter the information for a lookup identifier as described in the previous step (scroll down if necessary).

    d. Select **Save**.

Note: The **Allow null attribute** option in the hybrid option, is set to **false**. Therefore, all of the selected criterion attributes from both the currently selected table and the lookup table, must have a value. Also, setting optional conditions is available only for the lookup table, and is not available for the main table.

11. (Optional) On the Related Entries section select an existing related entry to edit, or select **Add** to create one.

    a. Update the Related Entry form and then select **Save**.

Related Entry form

Field	Description
Active	Check box that specifies that the related entry is active.
Related table	A related table that references the CI that is being matched.
Referenced field	A referenced field in **Related table** that should store the referenced CI. This field always references the cmdb_ci table, or a descendent of the cmdb_ci table.
Priority	Priority of the related entry for the specified **Related table**. Rules with lower priority numbers are given higher priority while matching a related item for specific related table. Related entries for the specified related table with identical priorities are applied randomly. You can keep gaps between the priority numbers, so you can assign the unused priority numbers to new entries without

Field	Description
	modifying the existing priority order.
Criterion attributes	The set of attributes to uniquely identify the related item. Attributes can belong to the current class, or to a parent class. **Note:** It is possible to add reference fields as a criterion attribute. However, such fields might not always be effective: • Reference fields store sys_ids that point to a record in another table, and thus is considered a weak criterion attribute (in terms of uniqueness) for the current table. • The system detects and then replaces invalid values in a reference field with 'Unknown'. For example, an invalid Model ID value is replaced with the value 'Unknown'. Also, if several CIs end up having that same reference field set to 'Unknown', then these CIs become duplicate CIs.

Field	Description
	Select the lock icon to view, add, or remove attributes from the identification rule.
Allow null attribute	If at least one criterion attribute in the related table is not null, allow to attempt matching with an identifier entry even if there are criterion attributes which are null.
Filter conditions	Add conditions to construct a filter to narrow the set of records that will be searched for a matching related item.

Note: If criterion attributes have only two attributes and sys_class_name is one of them (for example [name, sys_class_name], [ip_address, sys_class_name]), then the other attribute cannot be NULL, even if **Allow null attribute** is enabled. This restriction is due to sys_class_name being considered a special system matching attribute.

Example

For example, the pre-defined **Hardware Rule** applies to the Hardware [cmdb_ci_hardware] table. It has an identifier entry with the criterion attribute **Serial Number**, **Serial Number Type** and its **Search on table** field is set to **Serial Number**.

The following payload snippet adds a CI to the cmdb_ci_linux_server class, that is a child of the Hardware class. It also shows how you can add related items in the payload for which you should create **Related Entries** on the CI Identifier page for the Hardware [cmdb_ci_hardware] table:

```
{
    "items": [
        {
            "className": "cmdb_ci_linux_server",
```

```
            "lookup": [
                {
                    "className": "cmdb_serial_number",
                    "values": {
                        "serial_number": "VMware-42 21 e
3 da 44 14 5a a6-56 48 2b 0a 28 53 42 4c",
                        "serial_number_type": "system",
                        "valid": "true"
                    }
                },
                {
                    "className": "cmdb_serial_number",
                    "values": {
                        "serial_number": "4221E3DA-4414-5
AA6-5648-2B0A2853424C",
                        "serial_number_type": "uuid",
                        "valid": "true"
                    }
                },               }
            ],

    “related": [
                {
                    "className": "cmdb_ci_ucs_chassis",
                    "values": {
                        “name": “chassis1",
                        “category": “category1",
                        “short_description": “My Chassis
1"
                    }
                },
                {
                    "className": "cmdb_ci_ucs_chassis",
                    "values": {
                        “name": "chassis2",
                        "category": "category2",
                        "short_description": "My Chassis
2"
                    }
                },    }
     ],

            "values": {
                ......
```

```
                "name": "xpolog2.lab3",
                "os_name": "Linux",
                "output": "Linux xpolog2.lab3 2.6.32-431.
el6.x86_64 #1 SMP Fri Nov 22 03:15:09 UTC 2013 x86_64 x86
_64 x86_64 GNU/Linux",
                "serial_number": "VMware-42 21 e3 da 44 1
4 5a a6-56 48 2b 0a 28 53 42 4c",
                "sys_class_name": "cmdb_ci_linux_server"
            }
        }
    ]
}
```


When the **Hardware Rule** is applied, the Serial Number [cmdb_serial_number] table is searched for a match with the values specified within the `lookup` key. Unless **Enforce exact count match (Lookup)** is checked, it is not necessary for every lookup key to return a match, as long as there is at least one match. If all matches reference the same CI, then that CI is considered to be the existing CI record. If no match is found, then the identification search continues to the next rule entry. If after all the rules are exhausted without finding a match, a new CI record is created in the database.

What to do next

You can optionally create an inclusion rule to narrow the scope of CIs that are included in identification.

Related tasks

- Create an identification inclusion rule

Related concepts

- General guidelines for using CMDB Identification

Create an identification inclusion rule

Narrow the scope of CIs that are included in the identification process by creating an identification inclusion rule.

Before you begin

Role required: itil has read access, itil_admin (on top of itil) has full access.

About this task

During duplication detection of independent CIs, the identification and reconciliation engine (IRE) processes only the CIs that satisfy the identification inclusion rules. For example, you can set a filter to include only CIs whose state is operational. When no identification inclusion rules exist, all CIs are included in the identification process and in the CMDB Health duplicate metric calculations. In the base system, there are no predefined identification inclusion rules. Identification inclusion rules are defined at the class level.

Identification inclusion rules also indirectly impact what appears in CMDB health dashboards for duplicate CIs, in addition to any health inclusion rules.

> **Note:** Identification inclusion rules impact any script that calls IRE, therefore create them carefully. Identification inclusion rules can prevent the identification of certain types of CIs, affecting some features of Discovery and Service Mapping.

Procedure

1. Navigate to **All > Configuration > CI Class Manager**.
2. Select **Hierarchy** to display the CI Classes list.
3. Select the class for which to create an identification inclusion rule.
4. In the class navigation bar, expand **Class Info** and then select **Identification**.
5. Create or edit a rule and specify its criteria.
    a. In the Inclusion Rule (Advanced) section, select **Add** to create a rule or select **Edit** to edit an existing rule.
    b. In the Create Inclusion Rules dialog box, specify a criteria in the **Active record condition** field.

CIs must meet this criteria to be included in the identification process and in the duplicate CMDB Health metric.

6. Select **Save**.

What to do next

Navigate to **All > Configuration > Identification/Reconciliation > Identification Inclusion Rules >** to see the list of all identification inclusion rules.

Related tasks

- Create a CI identification rule
- Create health inclusion rule

Related concepts

- General guidelines for using CMDB Identification

General guidelines for using CMDB Identification

Review the following general guidelines for using CMDB Identification effectively.

Identification rules

An independent identification rule identifies a CI based on the CI's attributes, independently of other CIs.

A dependent identification rule identifies a CI by its dependent CIs and the relationships of the identified CI with those dependent CIs. Identification with a dependent identification rule is based on the dependent CIs and the relationships and qualifiers between the identified CI and its dependent CIs. Identification then requires more time than with an independent identification rule and is prone to some identification errors. Usage of dependent rules should therefore be minimized.

CI modeling determines which type of identification rules are required for proper CI identification.

Create identification rules using the following order of importance:

1. Independent identification rules — It is always preferable to create independent identification rules rather than dependent identification rules. When you model a CI, define the CI with a complete set of attributes that lend themselves to independent identification, eliminating the need to use additional CIs for identification.
2. Dependent identification rules — If it is necessary to create dependent identification rules, then define a single level of dependency. Two is the maximum number of dependency levels that is supported.
3. Avoid creating lookup identifier entries. The use of lookup identifier entry is highly discouraged as it can reduce performance. If unavoidable, ensure to first review class definitions and consider updates that allow usage of independent identification rules.
4. Limit the number of identifier entries within an identification rule, ideally to 1. A second identifier entry can further reduce performance, as will each additional identifier entry.
5. Create strong identification rules in which the strongest identifier entries and related entries are set with the highest priority.
6. Ensure that the identification rule is at the class level that it needs to be.

Payload

Create the payload using the following order of importance:

1. Payload size — Limit the number of CIs per payload to 500.
2. Avoid duplicate entries in the payload.
   Example: If an identification rule has a criterion attribute for the **name** field, then the following payload has duplicate items resulting in failure:

```
var payload = {
    items: [{
        className:'cmdb_ci_linux_server',
        values: {
            name:'Win Server 200',
            ram:'2048'
```

```
        }},
{
        className:'cmdb_ci_linux_server',
        values: {
            name:'Win Server 200',
            ram:'4096'
        }}]
};
```

3. Do not pass system data such as the following in the payload.

```
var payload = {
    items: [{
        className:'cmdb_ci_linux_server',
        values: {
            name:'Win Server 200',
            sys_domain:'global',
            sys_domain_path:'xyz',
            sys_updated_on:'2017-06-15 16:25:11',
            sys_mod_count:23,
        }}]
};
```

4. Provide the minimum necessary set of criterion attributes for each payload item, according to what is specified in the corresponding identification rules.

5. When matching CIs, use CIs' sysIds if available. If provided, IRE can use the sysId to directly locate a CI without requiring any criterion attributes from the identification rule. In this case, IRE does not use the sysId in the matching process.

    - Example: Independent CI that needs to be updated — sysId is available.

```
var payload = {
    items: [{
        className:'cmdb_ci_linux_server',
        values: {
            sys_id:'194876usytrr65378098',
            ram:'2048',
        }}]
};
```

- Example: Dependent CI that needs to be inserted. Tomcat War CI depends on Tomcat CI, and Tomcat CI depends on Linux Server CI. SysIds for the Tomcat and the Linux CIs are available.

```
var payload = {
        items: [{
            className:'cmdb_ci_app_server_tomcat_wa
r',
            values: {
                name:'war1',
                short_description:'my description
'             }
        }, {
            className:'cmdb_ci_app_server_tomcat',
            values: {
                sys_id:'194876usytrr65378098'
            }
        }, {
            className:'cmdb_ci_linux_server',
            values: {
                sys_id:'09876tysueyt6345lakiu'
            }
        }],
        relations: [{
                    parent:1,
                    child:0,
                    type: 'Contains::Contained by'}
, {
                        parent:1,
                        child:2,
                        type:'Runs on::Runs'}
                    ]
                };
```

- Example: Dependent CI that needs to be updated — sysId is available.

```
var payload = {
        items: [{
            className:'cmdb_ci_app_server_tomcat_wa
r',
            values: {
                sys_id:'039387euey637465sytet',
                short_description:'my description n
```

```
ew'                }
        }]
                  };
```

6. When inserting many CIs, all of which depend on the same CI, you should serialize your API calls. Otherwise, attempting to concurrently process many CIs can clog the system, significantly degrading overall system performance.

Related tasks

- Create a CI identification rule
- Create an identification inclusion rule

Reconciliation rules

Reconciliation rules determine which discovery sources can update CI attributes.

Discovery sources, such as EventManagement, ImportSet, ManualEntry, and Tivoli, are used with the createOrUpdateCI() API to simulate manual updates to CIs. Without reconciliation rules, discovery sources can overwrite each other's updates to attribute values.

There are two types of reconciliation rules:

Static reconciliation rules

Static reconciliation rules are the legacy reconciliation rules that set priorities for the various discovery sources for updating CI attributes. Static reconciliation rules specify which discovery sources can update class attributes, and the precedence order among these discovery sources.

When creating static reconciliation rules, ensure that there is a reconciliation rule for each discovery source that is authorized to update an attribute. Reconciliation rules can be defined at the parent and the child class level.

Static reconciliation rules are stored in the Reconciliation Definition [cmdb_reconciliation_definition] table.

Dynamic reconciliation rules

Dynamic reconciliation rules are based on attribute values processed by CMDB 360/Multisource CMDB rather than on discovery source priority. First, CMDB 360 processes the current payload data into the CMDB 360 data store. Then, applying a dynamic reconciliation rule, IRE selects the largest or most reported value, for example, across all discovery sources. Because dynamic reconciliation rules leverage CMDB 360, you must enable that feature to use dynamic reconciliation rules.

Creating dynamic reconciliation rules can be useful, for example, if it becomes difficult to set priority order for multiple discovery sources. Only a single dynamic reconciliation rule can exist per class attribute.

Dynamic reconciliation rules are stored in the Dynamic Reconciliation Definitions [cmdb_dynamic_reconciliation_definition] table.

Examples of static reconciliation rules

The following sample static reconciliation rules are created for the cmdb_ci_computer class and its cmdb_ci_linux_server child class:

1. Discovery is exclusively authorized to update the name attribute in the cmdb_ci_computer class.

   Because reconciliation rules are derived by child classes from parent classes, this rule also authorizes Discovery to update the name attribute in any child classes for the cmdb_ci_computer class.

2. ServiceWatch is exclusively authorized to update the name attribute in the cmdb_ci_linux_server class.

3. ServiceWatch is exclusively authorized to update all attributes in the cmdb_ci_linux_server class, as configured by leaving the **Attributes** field empty in the rule.

See Create a CI reconciliation rule for details about creating a static reconciliation rule that, for example, authorizes a discovery source to update a specific attribute such as name.

Using reconciliation rules

As you create reconciliation rules, keep in mind the following principles which are designed for flexibility and the refinement of rules at the attributes level:

Precedence of dynamic reconciliation rules

When both, static and dynamic reconciliation rules exist for the same CI attribute, the dynamic reconciliation rule takes precedence over the static reconciliation rule.

Authorization for all attributes in a class

A static reconciliation rule lets you authorize a discovery source to update all attributes in a class. However, this authorization can be overridden for some of the attributes by rules for child classes in which specific attributes are listed.

For example, if only example rules #1 and #3 above are created, then Discovery is authorized to update the name attribute in the cmdb_ci_linux_server class. ServiceWatch is authorized to update all other attributes in the class except for the name attribute.

To override the authorization of Discovery to update the name attribute, example rule #2 above is added to specifically authorize ServiceWatch to update the attribute.

Authorization to only specific attributes in a class

To authorize a discovery source to update specific attributes in a class, create a static reconciliation rule for the discovery source, and list these attributes in the rule. A rule that grants access to specific attributes in a class overrides other static reconciliation rules with an empty attribute list that grants access to the entire class.

Example rule #1 above grants Discovery with exclusive authority to update the name attribute of the cmdb_ci_computer class. All other discovery sources are prevented from updating the name attribute of any CI in the cmdb_ci_computer class.

Child class rules overrides parent class rules

Any reconciliation rules defined for a child class override the rules defined for its parent class. This rule applies also when the child's reconciliation rule is static and the parent's rule is dynamic (dynamic reconciliation rules have precedence over static reconciliation rules when they are for same level class).

For example, rule #1 above lets Discovery update the name attribute in the cmdb_ci_computer class and all of its child classes. However, rule #2 for the cmdb_ci_linux_server child class, which overrides rule #1 for the parent class, explicitly authorizes ServiceWatch to update this attribute in the child class.

As a result:

- Discovery cannot update the name attribute of the child cmdb_ci_linux_server class. Only ServiceWatch is authorized to update this attribute.
- Discovery is authorized to update the name attribute of CI records in all other child classes of the cmdb_ci_computer class.

Overlapping static reconciliation rules

Static reconciliation rules that authorize different discovery sources for the same attributes of the same class can coexist and do not exclude each other.

For example, assume the following rule is added. It is similar to example rule #1 above but authorizes a different discovery source:

ServiceWatch is authorized to update the name attribute in the cmdb_ci_computer class.

Like example rule #1 above, this new rule applies to the name attribute in the cmdb_ci_computer class so both Discovery and ServiceWatch can update the attribute. Any reconciliation rules are enforced to prevent the discovery sources from overwriting each other's updates.

For more information about reconciliation rules, see the [CMDB - Data Precedence Rules] Understanding the CMDB data precedence rules and troubleshooting [KB0756709] knowledge base article (Starting with the Paris release, reconciliation and data precedence rules are merged.

Domain separation

If Domain Separation is enabled, then you can scope reconciliation rules to specific domains. Rules of the parent domain, if not overridden, apply to CIs of child domain. All rules that are visible to a domain are applied, and a rule overriding the parent domain displays thele child domain version.

Understanding the CMDB reconciliation rules and troubleshooting [KB0756709]

- Create a CI reconciliation rule

  Create a static or a dynamic CI reconciliation rule.

- Create a data refresh rule

  Specify data refresh rules to determine if a CI is stale for a specific discovery source. Such CIs can then be updated by a lower-priority authorized discovery source.

Create a CI reconciliation rule

Create a static or a dynamic CI reconciliation rule.

If both, static and dynamic reconciliation rules exist for the same CI attribute, the dynamic rule has precedence.

> **Note:** You can't create a reconciliation rule for system fields or for Identification and Reconciliation Engine (IRE) specific fields such as the Discovery source (discovery_source) field. Also, reconciliation rules can't be dot-walked using reference fields.

Related tasks

- Create a data refresh rule

Create a static reconciliation rule

A static reconciliation rule specifies class attributes that discovery sources are authorized to update, and prevents unauthorized discovery sources from overwriting the attributes' values. A static reconciliation rule also

specifies the prioritization among multiple discovery sources. Without static reconciliation rules, discovery sources can overwrite each other's updates to attribute values.

Before you begin

Role required: itil has read access, itil_admin (on top of itil) has full access.

About this task

Static reconciliation rules are used in conjunction with data refresh rules to determine reconciliation steps for a CI. These rules determine if, when, and by which discovery source a CI can be updated. If multiple discovery sources are authorized to update the same class attributes, assign a priority to each of these discovery sources to prevent them from overwriting each other's updates.

After an authorized discovery source updates an attribute, subsequent updates are accepted only from the same discovery source or from a discovery source with a higher priority. Updates from a discovery source with a lower priority are rejected, unless these two conditions are met:

- The lower priority source is the first source updating the CI.
- The CI became stale based on data refresh rules for the CI class. However, a reconciliation rule that applies to all attributes, doesn't have precedence over a lower priority reconciliation rule that applies to a specific attribute, even if the CI is stale.

Precedence order of static reconciliation rules:

- Rule configured for a specific attribute, has precedence over rule set with **Apply to all attributes** (regardless of priority value).
- Between two rules for the same attribute or between two rules set with **Apply to all attributes**, the rule that is specific directly for the class has precedence over the derived rule.
- Between two rules for the same attribute or between two rules set with **Apply to all attributes** at the same class level, precedence is determined by rule priorities.

Information about the last discovery source that updates each attribute is stored in the Data Source History [cmdb_datasource_last_update]

table, but only after enabling the reconciliation rule. Therefore, there might be unexpected updates after you enable the rule until the highest priority data source has updated the CI.

Static reconciliation rules affect reconciliation of stale CI attributes. During reconciliation, the information in the Data Source History table is considered along with the data refresh rules for the CI's class, to determine if a CI attribute is stale. A CI attribute is determined to be stale if it was not updated by the latest discovery source to update the CI, within a time period. The time period is specified by the Effective Duration time in the data refresh rule for the class for the discovery source. In this case, if another authorized discovery source, with a lower priority attempts to update the stale CI attribute, the update is allowed.

If there is a dynamic reconciliation rule for the same CI attribute as in a static reconciliation rule, the dynamic rule takes precedence.

Procedure

1. Navigate to **All > Configuration > CI Class Manager**.
2. Select **Hierarchy** to open the CI Classes hierarchy list.
3. Select a class for which to create a reconciliation rule.
4. In the class navigation bar, expand **Class Info** and then select **Reconciliation Rules**.
5. In the Reconciliation Rules section, select **Add** to create a rule or select an existing rule to edit.
6. Select the **Static Reconciliation Rule** tile if it appears.
   If CMDB 360/Multisource CMDB is not enabled, you can't create a dynamic reconciliation rule and the tiles to choose the rule type do not appear.
7. Fill out the fields on the **Add Data Sources & Prioritize** tab, and then select **Next**.

Field	Description
Active	Check box to activate this reconciliation rule.

Field	Description
Discovery Source	Discovery source that you are configuring this rule for.
Priority	Priority of **Source** within other discovery sources for the specified attributes. Smaller numbers designate higher priority. Discovery sources without a reconciliation rule are assigned the lowest priority.

You can add multiple pairs of **Discovery Source** and **Priority**.

8. Fill out the fields on the **Select Attributes** tab, and then select **Next**.

Field	Description
Apply to all attributes	Authorizes the specified discovery sources to update all attributes of the specified class. **Note:** This rule will be overridden by any rule that applies to a specific attribute. In which case, instead of using this option, you can directly include all attributes for **Attributes**.
Attributes	Attributes, from the current or from a parent class, that the specified discovery sources are authorized to update. Available only if **Apply to all attributes** is not selected.

Field	Description
Update with Null	Attributes that the specified discovery sources can update with a null value. By default, authorized discovery sources cannot overwrite a non-null value with a null value. Attributes in this list, which are not in the **Attributes** list, are not included with the attributes that the discovery sources can update with a null value.

9. Fill out the fields on the **Set Filter Condition** tab, and then select **Save**.

Field	Description
Filter Condition	Conditions that CIs must meet for the rule to be applicable. For example, to apply this rule only to CIs that are associated with the Finance department, select this condition: **[Department] [is] [Finance]**.

Note: The glide.identification_engine.enable_reconciliation_filter_before_update system property determines when filter conditions are applied. By default, those filter conditions are applied after attribute values have changed during payload processing. Set this property to **true** so that Identification and Reconciliation Engine (IRE) applies the filter conditions before attribute values change.

What to do next

- Select the filter icon ( ) and select:
  - **Attributes**: To show only reconciliation rules for a specific attribute.
  - **Discovery sources**: To show only reconciliation rules for a specific discovery source.
- Select **Preview Rule** to see per attribute, the precedence order between any discovery sources that are authorized to update that attribute and any dynamic reconciliation rules.
- If CMDB 360/Multisource CMDB is enabled, you can:
  - Select **Preview Data** to see all attributes for a specific CI. Also, for each attribute, the current CMDB value and discovery sources reported values for the attribute.
  - Select **Recompute** to recompute CI attribute values after changing reconciliation rules.
- Navigate to **All > Configuration > Identification/Reconciliation > Reconciliation Definitions** to see a list view of all definitions of reconciliation rules.

Create a dynamic reconciliation rule

A dynamic reconciliation rule uses CMDB 360 data to choose a value such as the largest value that is reported, for updating a CI.

Before you begin

CMDB 360/Multisource CMDB must be enabled.

Role required: itil has read access, itil_admin (on top of itil) has full access

About this task

If the same CI attribute has both, a static reconciliation rule and a dynamic reconciliation rule, the dynamic reconciliation rule has precedence.

A dynamic reconciliation rule supports several rule types, such as largest reported value and most reported value. When applying a dynamic reconciliation rule, IRE processes the current payload and then examines the CMDB 360 data store to select a value with which to update the CMDB. Depending on the dynamic reconciliation rule type, selecting the appropriate value might not be immediately conclusive. For example, there might not be a single value that is most reported, or for some values, the last discovered timestamp isn't reported. Therefore, when necessary, IRE falls back to examining additional details such as last reported, last discovered, and last updated values to select the most appropriate value.

> **Note:** You can't add a dynamic reconciliation rule when creating a new child class in the CI Class Manager. You must first save the new child class and then add the dynamic reconciliation rule.

Procedure

1. Navigate to **All > Configuration > CI Class Manager**.
2. Select **Hierarchy** to open the CI Classes hierarchy list.
3. Select a class for which to create a reconciliation rule.
4. In the class navigation bar, expand **Class Info** and then click **Reconciliation Rules**.
5. In the Reconciliation Rules section, click **Add** to create a rule or select an existing rule to edit.
6. Select the **Dynamic Reconciliation Rule** tile.
7. On the **Select Rule** tab, select a rule type in the **Dynamic Rule Type** list field, and then click **Next**.
8. On the **Select Attributes** tab, select the attributes for which to apply the rule.
9. Select **Next**.
   Attributes that the specified rule type can't be applied to and attributes for which a dynamic reconciliation rule already exists for, don't appear.
10. Fill out the fields on the **Set Filter Condition** tab, and then select **Save**.

Field	Description
Filter Condition	Conditions that CIs must meet for the rule to be applicable. For example, to apply a rule only to CIs that are associated with the Finance department, select this condition: **[Department] [is] [Finance]**.

What to do next

- Select the filter icon ( ) and select:
  - **Attributes**: To show only reconciliation rules for a specific attribute.
  - **Discovery sources**: To show only reconciliation rules for a specific discovery source.
- Select **Preview Rule** to see per attribute, the precedence order between any discovery sources that are authorized to update that attribute and any dynamic reconciliation rules.
- Select **Preview Data** to see all attributes for a specific CI. Also, for each attribute, the current CMDB value and discovery sources reported values for the attribute.
- Recompute CI attribute values.
- Navigate to **All > Configuration > Identification/Reconciliation > Reconciliation Definitions** to see a list view of all definitions of reconciliation rules.

Create a data refresh rule

Specify data refresh rules to determine if a CI is stale for a specific discovery source. Such CIs can then be updated by a lower-priority authorized discovery source.

Before you begin

Role required: itil has read access, itil_admin (on top of itil) has full access.

About this task

Data refresh rules are used in conjunction with static reconciliation rules to determine reconciliation steps for a CI. These rules determine if, when, and by which discovery source a CI can be updated. The precedence order of applying reconciliation rules to a class, remains the same even when there are data refresh rules for that same class.

Data refresh rules have no impact when dynamic reconciliation rules are in effect.

Procedure

1. Navigate to **All > Configuration > CI Class Manager**.
2. Select **Hierarchy** to show the CI Classes list and then select the class for which to create a data refresh rule.
3. In the class navigation bar, expand **Class Info** and then click **Reconciliation Rules**.
4. Create a rule by selecting **Add** in the Data Refresh Rules section, or select an existing rule to edit. Then fill out the details in the Create Data Refresh Rules dialog box.

Field	Description
Discovery source	Discovery source for which staleness is evaluated.
Effective Duration	The time period that is used for the staleness test. If the fields specified in the static reconciliation rule for the CI's class weren't updated by **Discovery source** within the specified **Effective Duration** —

Field	Description
	the CI is determined to be stale for **Discovery source** and a lower priority discovery source is authorized to update the CI. When set to 0 (default), the CI is determined to be stale for **Discovery source** immediately after an update and therefor, a lower priority discovery source will always be authorized to update the CI. If you enter a value with a prefix that is valid and a suffix that is not, such as `15 x` — the valid portion of the value is used ('15'). If the entire value is invalid — the default value of 0 is used.
Active	Activates the rule.

5. Click **Save**.

Related concepts

- Create a CI reconciliation rule

Configure CI reclassification during IRE processing

During the Identification and Reconciliation Engine (IRE) CI identification process, a CI might need to be reclassified to a different sys_class_name type. By default, CIs are reclassified automatically. If automatic reclassification is disabled, then the CI is not reclassified and the system generates a reclassification task for your review.

The class of a CI can be upgraded, downgraded, or switched to a different branch in the class hierarchy. For more details about reclassification operations, see Reclassify a CI. You can use system

properties and payload flags to configure the IRE behavior of CI reclassification, globally or individually per CI.

> **Note:** CI reclassification is possible only between two classes that have identical identification rules.

Configure automatic CI reclassification using system properties

You can use system properties to configure system-wide IRE behavior for CI reclassification. For information about CI reclassification-related properties, including access, see Properties.

- The following properties enable or disable automatic reclassification updates that are specified in a payload. These properties are set to **true** in the base system, enabling processing of CI updates, including CI reclassification updates.

  To disable any automatic reclassification update, set the respective property to **false**. In that case, IRE rejects a payload (or a payload item in Enhanced IRE) with the respective reclassification updates, and creates a reclassification task.

  - glide.class.upgrade.enabled
  - glide.class.downgrade.enabled
  - glide.class.switch.enabled

- The following properties enable IRE to process CI updates with reclassification operations. However, depending on the property setting, IRE processes or skips the reclassification update. These properties are set to **false** in the base system, in which case IRE processes CI updates including any CI reclassifications.

  Set a property to **true** to configure IRE to process CI updates but not the CI respective reclassification update.

  - glide.identification_engine.update_without_switch_enabled
  - glide.identification_engine.update_without_downgrade_enabled

- glide.identification_engine.update_without_upgrade_enabled

This set of properties takes precedence over the previous set of properties (glide.class.<reclassification>.enabled). For example, with the following conflicting property settings, the second property takes precedence over the first:

- glide.class.downgrade.enabled = **false**
- glide.identification_engine.update_without_downgrade_enabled = **true**

Example for IRE processing of a payload item with a switch of a CI from Linux Server to Window Server. With the following default property settings in the base system, IRE updates the attributes including the class switch:

- glide.class.switch.enabled = **true**
- glide.identification_engine.update_without_switch_enabled = **false**

However, with the following property settings, IRE updates the attributes but skips the class switch:

- glide.class.switch.enabled = **true**
- glide.identification_engine.update_without_switch_enabled = **true**

Configure automatic CI reclassification in input payloads

You can use flags which correspond to the system properties, in the input payload of the CreateOrUpdateCIEnhanced() or the createOrUpdateCI() APIs. In the payload, set these flags to **true** or **false** to temporarily override the respective system property settings, at the payload item level.

For the following payload flags that control reclassification behavior, if any is set, the setting has precedence regardless of the setting of the corresponding glide.class.xxx.enabled property:

- `classUpgrade`
- `classDowngrade`
- `classSwitch`

For the following payload flags that control reclassification behavior, the system checks if either the flag or its corresponding glide.identification_engine.update_xxx_enabled property is true to allow the update without the respective reclassification operation:

- `updateWithoutUpgrade`
- `updateWithoutDowngrade`
- `updateWithoutSwitch`

Also, you can pass payload level settings (which apply to all items within a payload), per data source, by specifying CI reclassification properties on the Robust Import Set Transformers form. For more information, see Robust import set transformer properties.

The following sample JSON payload enables automatic reclassification for the specified CI:

```
{ items: [{className: 'cmdb_ci_server', classUpgrade: tru
e, classDowngrade: true, classSwitch: true, values: {name
: 'linux123', serial_number: '12srt567', ip_address: '10.
2.3.4'}, }]}
```

Reclassification restriction rules

Prevent IRE from downgrading or switching a CI class during payload processing to help prevent data loss. A reclassification restriction rule prevents a CI class change for specific source and target classes, while still processing any other property updates for the CI.

You can use a reclassification restriction rule, for example, to prevent a CI class downgrade from cmdb_ci_linux_server (source class) to cmdb_ci_server (target class). Or, to prevent a CI class switch from Linux Server to Windows Server. Reclassification restriction rules can be useful when using a Service Graph Connector which might lead to a class downgrade or switch, and a potential loss of important data.

To control the application of reclassification restriction rules:

- Use the glide.identification_engine.reclassification_restriction_rules_enabled system property to globally enable or disable the application of active reclassification restriction rules. This property is set to **true** by default.

- Use the skipReclassificationRestrictionRules payload flag in an IRE payload to prevent the application of active reclassification restriction rules.

  For example, a payload with the skipReclassificationRestrictionRules flag:

```
{
  "items": [
    {
      "className": "cmdb_ci_server",
      "values": {
        "short_description": "Linux server description",
        "name": "Linux Server 1"
      },
      "settings": {
        "skipReclassificationRestrictionRules": "true"
      }
    }
  ]
}
```

For information about how to create a reclassification restriction rule, see Create a reclassification restriction rule.

Create a reclassification restriction rule

Reduce data loss during IRE processing by preventing a CI class change for specific source and target classes. A reclassification restriction rule affects only the Class attribute and does not prevent the update to the rest of the CI properties.

Before you begin

Role required: Itil_admin (Itil has read privilege only)

About this task

If during IRE processing of a payload, a CI needs to be reclassified (downgrade or switch class), IRE checks reclassification restriction rules. If any reclassification restriction rule applies to the current CI

reclassification, IRE processes the CI properties update, but skips the CI reclassification.

IRE output provides specific details about any processing related to reclassification restriction rules.

A reclassification restriction rule applies only to the direction between the specified source and the target classes. The rule doesn't prevent a reclassification in the opposite direction, from the specified target class to the source class. To restrict reclassification between two classes in both directions, specify two separate reclassification restriction rules, one for each direction.

Procedure

1. Enter `cmdb_ire_reclassification_restriction.list` in the filter navigator.
2. Fill out the Reclassification Restriction form.

Field	Description
Name	Name of the reclassification restriction rule.
Source table	Current CI class.
Source inheritance	Whether to apply the reclassification restriction rule to child classes of **Source table**.
Target class	Reclassification target class.
Target inheritance	Whether to apply the reclassification restriction rule to child classes of **Target table**.
Type	CI reclassification type: **Downgrade** or **Switch**.

3. Click **Submit**.

What to do next

In the Reclassification Restrictions list view, you can activate or deactivate a reclassification restriction rule by setting its **Active** value to true or false.

Create an IRE data source rule

When using Identification and Reconciliation Engine (IRE), you can prevent a specific discovery (data) source from inserting new CIs for a specific class. Create IRE data source rules for discovery sources that you don't trust in creating CIs but continue to trust in updating those CIs that exist.

Before you begin

Role required: itil_admin

About this task

IRE data source rules have no impact when dynamic reconciliation rules are in effect.

For example, an IP scan tool that discovers network gear but does not discover servers and therefore creates server CIs without details. You can prevent such discovery source from creating specific CIs, while still permitting it to update those specific CIs if they exist. IRE data source rules are stored in the IRE Data Source Rule [cmdb_ire_data_source_rule] table.

- Child classes derive IRE data source rules from parent classes like identification rules do.
- IRE data source rules that are specified for a child class, override any IRE data source rules derived from a parent class.

When IRE processes an insert operation that is prohibited by an IRE data source rule, the insert operation fails. This failure happens when the discovery source and CI class in the insert operation and in an IRE data source rule, match. When CreateOrUpdateCIEnhanced() is used, IRE stores the failed payload in the CMDB IRE Partial Payloads [cmdb_ire_partial_payloads] table for future potential use.

Note: When an insert operation is not allowed by the IRE data source rule, then when using createOrUpdateCI(), the entire IRE payload fails since createOrUpdateCI() doesn't allow partial commits.

If later, a permitted discovery source attempts to insert that same CI, then IRE inserts the CI after merging it with the matching CI from the partial payloads. IRE then deletes the partial payload from the CMDB IRE Partial Payloads [cmdb_ire_partial_payloads] table, and allows future updates by the discovery source specified in the rule.

IRE data source rules do not apply to lookup and related items, and only a single rule can be active for any class/discovery source pair.

Procedure

1. Navigate to **All > Configuration > Identification/Reconciliation > IRE Data Source Rule**.
2. In the list view, click **New** and fill out the IRE Data Source Rule form.

Field	Description
Active	Activates the IRE data source rule.
Applies to	The class (and child classes) that the specified discovery (data) source is not allowed to create CIs of.
Data source	Discovery (data) source that is not allowed to create CIs of the specified class.
Insert Not Allowed	Disables the specified discovery (data) source from inserting new CIs from the specified class, to the CMDB.

3. Click **Submit**.

Result

If a payload item with an insert request, and in which the discovery source and the CI class match the discovery source and the CI class specified in the IRE data source rule:

1. The insert operation fails and IRE logs the following message:

   INSERT_NOT_ALLOWED_FOR_SOURCE Insert into [xyz] is blocked for data source [xyz] by IRE data source rule.

2. If using CreateOrUpdateCIEnhanced(), then IRE stores the payload item as a partial payload in the CMDB IRE Partial Payloads [cmdb_ire_partial_payloads] table.

If later, a permitted discovery source successfully inserts a CI that matches the CI from a partial payload item:

1. The current CI is merged with the matching CI from the partial payload, applying static reconciliation rules as needed.
2. The respective partial payload in the CMDB IRE Partial Payloads [cmdb_ire_partial_payloads] table is deleted.
3. Later payloads in which the non-permitted discovery source updates the respective CI, run successfully.
4. IRE allows the discovery source, that was previously prohibited from inserting the CI, to update that same CI which now exists in the CMDB.

CMDB dependent relationship rules

Service definitions consist of CI types and relationship types. Dependent relationship rules define the dependency structure of the CI types and the relationship types in these service definitions, helping in CI identification and in the construction of business service maps.

The dependencies that are defined by these rules are used when identifying dependent CIs to prioritize the order of CI identification, and to match CIs and respective dependent CIs in a payload. Dependent relationship rules are also used by Service Mapping and can be defined for custom CI types. After defining a new CI type, you can define

dependent relationship rules that specify how the new CI type is related to existing types in the CMDB.

Dependent relationship rules consist of hosting and containment rules (dependent relationship rules), each type modeling the data from a different perspective of the CI. Containment rules represent CIs' configuration hierarchy, describing which CI contains which other CIs. Hosting rules represent CIs' placement in a business definition, describing what CIs run on.

Both hosting and containment rules describe a relationship type between two CI types and the same relationship type can be used in a hosting rule and in a containment rule. It is the context in which the relationship is used that distinguishes between a containment and hosting rule.

Manage dependent relationship rules:

- To access rules at the class level, use the CI Class Manager. Navigate to **All > Configuration > CI Class Manager**.
- To access grouped rules, use the Metadata Editor. Navigate to **All > Configuration > Identification/Reconciliation > Metadata Editor**.

The plugins that have been activated on an instance determine which hosting and containment rules exist in a base system.

Hosting rules

Hosting rules represent all the possible valid combinations of pairs of hosting and hosted CIs in the service definition. Hosting rules are a flat set of rules that can be only one level deep, and which always involve resources, typically physical or virtual hardware. Each hosting rule is a stand-alone rule between two CI types, describing either a valid CI type that another CI type can host, or by which another CI type can be hosted. A hosting rule consists of a parent CI type, a relationship type (such as Hosted On::Hosts) and a child CI type. For example, you can have a hosting rule that specifies that the CI type ‘Application’ ‘Runs On::Runs’, the CI type ‘Hardware’.

A CI can be hosted on multiple resources (such as Windows and Linux). This CI is represented by a hosting rule for the CI with each resource that the CI can be hosted on. During CI identification, the pair of CIs that are being examined, should satisfy at least one hosting rule.

Hosting rules are stored in the CMDB Metadata Hosting Rules [cmdb_metadata_hosting] table.

Containment rules

Containment rules represent the containment hierarchy for a CI type, describing valid objects that a CI type can contain in the service definition, and valid objects that can be contained by the CI type. Containment rules are chained to each other in a containment rules group, with a CI type that is the top-level (root) parent of the group. The collection of containment rules construct a hierarchy-like map of containment relationships. Containment rules are logical concepts used to represent logical CIs, for example to describe software that runs on a server. A containment rule consists of a parent CI type, a relationship type (such as 'Contained By::Contains'), and a child CI type. For example, you might have a containment rule specifying that the CI type 'Tomcat' 'Contains::Contained By' CI type 'WAR File'.

Endpoints are special containment rules that specify incoming or outgoing connections in the model, designating the CI types that data of some specified type flows in to or out from the service definition. After adding an endpoint to a containment rule, you cannot add any child rules to the endpoint rule.

Containment rules are stored in the CMDB Metadata Containment Rules [cmdb_metadata_containment] table.

Reference rules

Reference rules are used mostly by Cloud Management to represent all of the possible valid combinations of pairs of referencing and referenced CIs in the service definition.

- Reference rules are a flat set of rules that can be only one level deep.
- Reference rules always involve resources, typically virtual entities. Each reference rule is a stand-alone rule between two CI types, describing either a valid CI type that another CI type can reference, or by which another CI type can be referenced. Both the CI classes should be able to live independent of each other.
- A referencing rule consists of a parent CI type, a relationship type (such as `Provisioned From::Provisioned`) and a child CI type. For example, you can have a referencing rule that specifies that the

CI type 'Virtual Machine' `Provisioned From::Provisioned`, the CI type 'Image'.

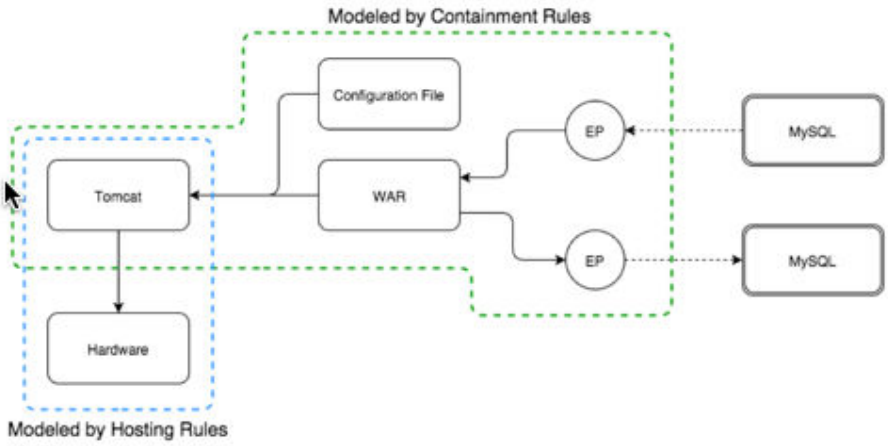
- A CI can reference multiple resources (for example, a VM Instance can have a reference relation with both the Image and the Hardware templates). This CI is represented by a referencing rule for the CI with each resource that the CI can be referenced from.
- The reference rule cannot be part of the CI identification.
- Reference rules are stored in the CMDB Metadata Reference Rules [cmdb_metadata_reference] table.

Rules requirements

The rules that you create are bound by the following requirements which narrow the relationships and ensure that only valid options are available in the drop-down lists in the Metadata Editor.

- Given a CI type that is as a child in a containment rule: Not this CI type or its children can be a top-level (root) parent of any other containment rule, and it cannot be in any hosting rule, either as a parent or as a child.
- Given a CI type that is a top-level (root) parent of a containment rule: It cannot be a child in a hosting rule (for example, you cannot be hosted on Tomcat, if Tomcat has any containment rules).
- Given a CI type that is a child in a hosting rule: It cannot be in any containment rule, either as a parent or a child.
- Given a CI type that is a parent in a hosting rule: It cannot be a child in any containment rule.
- Hosting rules cannot create loops such as Tomcat –runs_on- VMWare –runs_on- Tomcat.

Example: Hosting and containment rules model



Hosting rules that model the diagram:

Tomcat 'Runs on' Hardware.

Containment rules that model the diagram:

- Tomcat 'Contains' Configuration File
- Tomcat 'Contains' WAR
- WAR has two endpoints for JDBC with MySQL:
  - Inbound
  - Outbound

Example: Valid set of rules

```
Tomcat Hosted Linux
Linux Hosted Computer
```

The second metadata entry triggers the third requirement, which is satisfied (it is a hosting rule, not a containment rule).

- Create dependent relationship rules

Create hosting and containment rules (dependent relationship rules) for CI classes to help with correctly identifying dependent CIs during the business discovery process and service mapping. Discovery calls the identification API that applies dependent relationship rules.

Create dependent relationship rules

Create hosting and containment rules (dependent relationship rules) for CI classes to help with correctly identifying dependent CIs during the business discovery process and service mapping. Discovery calls the identification API that applies dependent relationship rules.

You can create a basic hosting or containment rule in the CI Class Manager. Or, use the Metadata Editor to create groups of hosting and containment rules, and inbound or outbound endpoints in containment rules. The CI Class Manager and the Metadata Editor are synchronized, and you can use each of those tools to display and edit a dependent rule.

Create a dependent relationship rule for a CMDB class

Use the CI Class Manager to create a basic dependent relationship rule (hosting or containment relationship rule) for a CMDB class.

Before you begin

Role required: itil has read access, itil_admin (on top of itil) has full access.

About this task

The class for which you create dependent relationship rule, must have a dependent identification rule.

Procedure

1. Navigate to **All > Configuration > CI Class Manager**.
2. Click **Hierarchy** to display the CI Classes list, and select the class for which you want to create a hosting or a containment rule.
3. In the class navigation bar, click **Dependent Relationships**.
4. In the Dependent Relationships view, click **Add dependency**.

5. Fill out the details in the Add Dependent Relationship Rule dialog box.

Field	Description
Rule Type	Designation of whether this rule is a hosting rule or a containment rule.
This Class	The class that the rule applies to.
Relationship	The relationship type for the rule.
Target Class	The target class for the dependent relationship rule. The designation of this class as a child or parent class, is based on the specified **Relationship**.

6. Click **Save**.

What to do next

You can click **Reset to derived** and then confirm the operation to delete all dependent relationship rules that were added specifically for the selected class. Only dependent relationships that are derived from a parent class, remain.

For more information about child and parent classes, see Table extension and classes.

Create or edit a collection of containment rules

Create containment rule for CIs to help with correctly identifying dependent CIs during the business discovery process and service mapping. Discovery calls the identification API that applies dependent relationship rules.

Before you begin

Role required: admin

About this task

A containment rule is a dependent relationship rule which defines a relationship between two CIs, structured as: CIType1 RelationshipType CIType2. The first CI type that you add becomes the top level CI of a containment rules group which is a chain of containment rules. The entire set of containment rules is organized as groups according to top-level CIs.

To create a containment rules group for a new CI type, you need to first add the CI Type1 of the relationship. To add a child containment rule for a CI type that exists, you need to select that CI type, and define the second portion of the relationship rule which is the relationship type and CI Type2.

To each rule within a containment rules group you can add inbound or outbound endpoints, which are noted by blue up and down arrows. After adding an endpoint, you can not add a containment rule in that branch of the containment rules hierarchy.

Procedure

1. Navigate to **All > Configuration > Metadata Editor**.
2. In the Metadata Editor, click the **Containment Rules** tab.
3. Click **Add New Rule** to add a top-level rule or point to a rule for which you want to add a child rule and click the green '+' icon that appears on the right.
4. Complete the **Add Containment Rule to <class>** form.

Field	Description
Configuration Item Type	The CI class that the rule applies to.
Relationship Type	The relationship type for the rule.
Reverse Relationship Direction	Enable to use the reverse relationship in the rule.

Field	Description
Always include in Service Model	Enable to always include the CIs of the specified class in the Service Map if their parent CI (based on the containment relationship) is present in the Service Map.

5. Click **Create**.
6. Add an endpoint to a child rule:
   a. Point to a child rule for which you want to add an endpoint.
   b. Click the blue "+" icon that appears on the right.
   c. Complete the **Add Endpoint To <class>** form.

Field	Description
Endpoint Type	The type of endpoint.
Inbound or Outbound	The direction of the endpoint.

   d. Click **Create**.

Create or edit a collection of hosting rules

Create hosting rule for CIs to assist in correctly identifying dependent CIs during the business discovery process and service mapping. Discovery calls the identification API that applies dependent relationship rules.

Before you begin

A hosting rule is a dependent relationship rule which defines a relationship between two CIs, structured as: <CI Type1> <relationship type> <CI Type2>. To create a hosting rule, you need to add a CI type as <CI Type1> in the relationship rule, and then define the second portion of the relationship rule which is the relationship type and <CI Type2>. The entire set of hosting rules is organized as groups according to the top-level hosted CIs.

A hosting rule implicitly contains two rules, which are the reversal of each other. When you create the rule '<CI Type1> <relationship type> <CI Type2>', the rule '<CI Type2> <reversed relationship type> <CI Type1>' is automatically added.

Role required: admin

Procedure

1. Navigate to **All > Configuration > Metadata Editor**.
2. In the Metadata Editor, click the **Hosting Rules** tab.
3. Click **Add New Rule** to add a top-level rule or point to a rule for which you want to add a child rule and click the green '+' icon that appears on the right.
4. Complete the **Add Hosted/Hosting Rule to <class>** form.

Field	Description
Configuration Item Type	The <CI Type2> in the rule.
Relationship Type	The relationship type for the rule.
Reverse Relationship Direction	Check to reverse relationship in the rule.

5. Click **Create**.

Applying IRE to Import Sets

You can apply CMDB Identification and Reconciliation Engine (IRE) processes when Import Sets are used to import CIs into the CMDB. CI identification can prevent duplicate CIs in the CMDB, which Import Sets might otherwise cause.

Populating CMDB tables using Import Sets can inadvertently result in duplicate CIs when multiple imported records are identical to an existing CI. To minimize this duplication, you can apply CMDB Identification and Reconciliation processes to Import Sets when importing new records into CMDB tables.

Transform map script

In the onBefore transform map script for an import set, add a call to the CMDBTransformUtil API, similar to the following code sample:

```
(function runTransformScript(source, map, log, target) {
// Call CMDB API to do Identification and Reconciliation o
f current row
var cmdbUtil = new CMDBTransformUtil();
cmdbUtil.setDataSource('ImportSet');
cmdbUtil.identifyAndReconcile(source, map, log);
ignore = true;

if (cmdbUtil.hasError()) {
        var errorMessage = cmdbUtil.getError();
        log.error(errorMessage);
} else {
        log.info('IE Output Payload: ' + cmdbUtil.getOutp
utPayload());
        log.info('Imported CI: ' + cmdbUtil.getOutputReco
rdSysId());
}

})(source, map, log, target);
```

The `ignore = true` code phrase prevents Import Sets from creating the same record again after it is processed by the identification engine.

Process

The identification engine performs identification of each source record before it is inserted into the CMDB. The identification engine determines if the record is a duplicate of an existing CI, and then:

- If not duplicate: Inserts the record to the target table.
- If duplicate: Updates the existing CI in the CMDB, with data from the source record.

The CMDBTransformUtil API pre-processes the source data, then passes the input values to the identification engine with import set being the data source by default. The CMDBTransformUtil API supports a target field that is a reference field in the same manner that Import Sets supports it. The CMDBTransformUtil API also supports a source script, evaluating

source scripts to determine the target value which is then passed to the identification engine. For more information, see Creating a field map.

Specify multiple target tables for an import set

You can configure each record in an import set with its own target table. Then, instead of inserting all the transformed records into a single target table, the records are inserted into the different target tables that are specified per record. For example, you might need to insert some records from the import set to the Computer class and other records to the Server class.

When importing data using Import Sets, incorporate the following steps:

- In the data source file, add a target table column. Use a string such as "MyTable" to label the column header. In each record row, enter the target table for the record, as a valid CMDB class name such as "cmdb_ci_computer".
- After you **Auto Map Matching Fields** on the Table Transform Map form, add a field map for the added target table column to establish a relationship between classes and the target tables in the CMDB.
    1. In the **Field Map** related list on the Table Transform Map form, click **New**.
    2. Set **Source field** to the header of the target table column that you added in the data source file, such as `MyTable`.
    3. Set **Target field** to **Class**.
    4. Click **Submit**.

When you configure an import set with multiple target tables as described in the steps above, the **Target table** that is specified on the Table Transform Map form is not used.

Restrictions

The following restrictions apply:

- An import set should be associated with a single transform map. While adding a call to the CMDBTransformUtil API, ensure that still a single transform map exists for the import set.

- The CMDBTransformUtil API does not check if mandatory fields have values when used with Import Sets . Regardless of how enforce mandatory fields is set in the transform map, data import fails if a mandatory field does not have a value.
- CI Identification and Reconciliation cannot be applied to Import Sets for dependent CIs (CIs with dependent identification rules).

Related topics

- Create a transform map

Detecting duplicate CIs

When IRE identification process detects duplicate CIs, it groups each set of duplicate CIs into a de-duplication task for review and remediation. A large number of duplicate CIs might be due to weak identification rules. You can configure the identification engine to reconcile duplicate CIs.

During Identification and Reconciliation Engine (IRE) processes, handling of duplicate CIs is determined by the properties glide.identification_engine.skip_duplicates (set to true by default) and glide.identification_engine.skip_duplicates.threshold (set to 5 by default), and on the number of duplicate CIs that are detected. You can configure these properties so duplicate CIs are automatically reconciled, skipping duplication.

- If glide.identification_engine.skip_duplicates is true, and the number of duplicate CIs is less than the threshold specified by glide.identification_engine.skip_duplicates.threshold, then the oldest of the duplicate CIs is picked as a match and gets updated. That oldest duplicate CI also becomes the main CI for that set of duplicate CIs. The rest of the duplicate CIs are tagged as duplicates by setting their duplicate_of attribute to the appropriate main CI. During matching, IRE filters out any CI that is tagged as duplicate of any CI.
- If glide.identification_engine.skip_duplicates is false, then matching of duplicate CIs fails with an error, and none of the duplicate CIs are updated.

Also, the glide.duplicate_ci_remediator.max.cis property determines de-duplication processing for a large number of duplicate CIs. For more

information, see the 'Large number of duplicate CIs' section in the Duplicate CIs remediation topic.

In either case, de-duplication tasks are always created.

> **Note:** For a duplicate CI, if any of the CI's attributes, other than duplicate_of, is updated by IRE processing, then the CI is no longer considered a duplicate CI. In that situation, the value of duplicate_of is cleared in the CI.

For more information about these properties, see Properties.

Remediating de-duplication tasks

For information about reviewing and remediating de-duplicate tasks, and how the main CI is used, see Duplicate CIs remediation.

Using identification simulation

Identification simulation is a central location for automatically constructing a payload that is guaranteed to be complete and valid. You can then simulate the processing of the payload by the Identification and Reconciliation Engine (IRE) and examine the results before actually submitting it for execution by IRE.

Use identification simulation to construct an input payload, and simulate processing of the payload by IRE. You can then examine the results, adjust identification rules if needed, and re-run the simulation of the updated payload.

Use the identification simulation to:

- Automatically construct input payload that is based on existing identification rules, hosting and containment rules.
- Simulate execution of a payload (automatically constructed by identification simulation, or manually created).
- Browse payload output and execution log messages for a simulated run.

Note:

- Identification simulation does not commit any updates to the CMDB.
- Identification simulation supports simulation of processing payloads that are provided and which contain non-CMDB tables, but doesn't support the generation of such payloads.

Automatically generate payload

Use identification simulation to automatically construct an input payload for a specified class. The constructed payload is complete with any required dependent CIs, correctly structured, and syntactically valid for processing by the identification and reconciliation engine (IRE).

Before you begin

Role required: admin

About this task

The payload that is constructed during identification simulation is for the specified class. For a dependent CI class, you will be prompted for information about all dependencies. After you provide the required details, identification simulation constructs the payload based on your input.

Note: Automatically generating payloads that contain non-CMDB tables, isn't supported.

Procedure

1. Navigate to **All > Configuration > Identification/Reconciliation**, and click **Identification Simulation**.
2. In the **Start with CI Class** box click **Start**.
3. On the **Payload Information** form, in the **Data source** field, select the data source that is associated with this class update.
   For the ServiceNow Discovery data source, select `ServiceNow`.

4. Select the **Class** in the payload.
    a. In the **Criterion Attributes** area select the CI identifier attributes and then specify the values that uniquely identify a CI.
    b. In the **Additional Attributes** area specify attributes and values that matching CIs will be updated with.
5. For dependent CIs associated with dependent identification rules, fill out the **Criterion Attributes** and **Additional Attributes** sections in all **Container level** sections that display.
6. Click **Generate Script**.
7. If any errors indicate that there are missing fields, fill in the missing fields and then click **Generate Script** again.

What to do next

- Click **Run Simulation** to simulate processing of the payload by IRE.
- Examine the results of the simulation, fine-tune the payload as needed, and combine with other payloads for other classes as desired. After finalizing the payload, use the createOrUpdateCI() API to execute the payload by IRE which will result in actual updates to the CMDB.
- Click **Copy Script** to copy the JSON script into the clipboard. You can then paste that script into a third party software or to another screen of the identification simulation.

Simulate payload processing using identification simulation

Use identification simulation to simulate the identification and reconciliation engine (IRE) process of CI identification for an input payload. Provide a valid payload, which was constructed using identification simulation or that was created manually.

Before you begin

Role required: admin

Procedure

1. Navigate to **All > Configuration > Identification/Reconciliation**, and click **Identification Simulation**.
2. (Optional) To run a simulation of an existing payload:
   a. Click **Start** in the **Start with Existing Payload** tile.
   b. On the Insert JSON Payload page, select the **Data source** that is associated with this class update.
   c. (Optional) Select **Use Enhanced Identification** to apply the identifyCIEnhanced API for enhanced CI identification, instead of using the identifyCI API.
   d. Paste the JSON payload into the empty canvas.
3. (Optional) To construct a new payload click **Start** in the **Start with CI Class** tile.
   See Automatically generate payload for more information.
4. Click **Run Simulation** to simulate processing of the payload by IRE.

What to do next

1. Examine the results of the simulation in the results pane, and fine-tune the payload as needed:
   a. Click **Run #1** to display the **Context ID** and the **Run ID** of the simulated run.
   b. Click the drop down arrow next to **Run #1** to display additional details.
      - **Input**: Displays the payload for the simulation.
      - **Logs**: Displays all the logged messages that IRE generated while simulating processing of the payload, according to the specified logging level.
      - **Output**: Displays the output payload returned by IRE.

2. After finalizing the payload, use the createOrUpdateCI() API to execute the payload by IRE which will result in actual updates to the CMDB.

Set logging level for identification simulation

Identification simulation logs each step of a simulated payload processing. You can then examine these run logs to determine if a payload was processed as expected, and if identification rules are effective. You can adjust the level of logging so it is helpful, and so that the amount of messages is not excessive or insufficient.

Before you begin

Role required: admin

Procedure

1. Navigate to **All > Configuration > Identification/Reconciliation**, and click **Identification Simulation**.
2. Click the **Settings** icon.
3. Select logging level for the identification and reconciliation engine (IRE) under **IE Log Level** and for the service cache under **Service Cache Log Level**.
   The logging levels are displayed in ascending order, from the minimum level to the maximum level of logging.
4. Click on the **Settings** icon again to close the **Settings** dialog box.

Examine run logs

Identification simulation provides run logs which are generated by Identification and Reconciliation Engine (IRE). You can access these run logs for payload runs, to examine results and for debugging purposes. IRE payload output logs appear in a user friendly format on a central page.

Before you begin

Role required: admin

About this task

Also, internal applications that use IRE (such as Discovery) can call an internal API to provide a URL to viewing IRE run logs.

Logging is in the context of a specific run of the identification engine, and you can filter the log list by a specific data source and time range. Up to 1000 run logs that are up to 2 months old are listed, grouped by Context IDs, and run times. You can use the glide.identification_logs.max_run_ids property to modify the 1000 limit.

You can control the logging level by using the glide.discovery.identification.log_level Discovery system property and setting the value to one of the following:

- Info
- Warn
- Error
- Debug
- DebugVerbose
- DebugObnoxious

> **Note:** IRE performs an initial verification of a payload before processing identification rules. If IRE detects any duplicate CIs based on any class identifiers, the payload is rejected and processing stops.

Procedure

1. Navigate to **All > Configuration > Identification/Reconciliation > Identification Logs**.
2. Filter the runs list as follows:
   a. **Source**: Select the data source for which to display run logs.
   b. **Time Range**: Specify a time range for which to display run logs.

The **Runs** list displays all runs for the specified data source, during the specified time range.

3. In the **Runs** list, click a **Run #** to display its **Context ID** and **Run ID**. A unique Context ID is associated with each specific payload that is run. Each run of that payload, is associated with a unique Run ID. A single Context ID for a payload that is run multiple times is associated with multiple Run IDs.
4. Click the drop down arrow for a **Run #** to display additional details.
    - **Input**: Displays the payload for the run.
    - **Logs**: Displays all the logged messages that the identification engine generated while running the payload, according to the specified logging level.
    - **Output**: Displays the output payload returned by the identification engine.

View a reclassification task

Reclassification tasks are created for CIs that couldn't be automatically reclassified during the identification process. Review these tasks to locate the CIs and decide if to reclassify them.

About this task

The properties you use to disable automatic CI reclassification determine whether reclassification tasks are created for those CIs that couldn't be automatically reclassified:

- Using any one of the 'glide.class.<reclassification operation>.enabled' properties (such as glide.class.upgrade.enabled): Reclassification tasks are created.
- Using any one of the 'glide.identification_engine.update_without_<reclassification operation>_enabled' properties (such as glide.identification_engine.update_without_switch_enabled): Reclassification tasks aren't created.

For more information about reclassification during IRE processing, see Configure CI reclassification during IRE processing.

Before you begin

Role required: admin or itil

Procedure

1. Navigate to **All > Configuration > Identification/Reconciliation > Reclassification Tasks**.
2. Select a reclassification task.
3. Examine the details on the Reclassification Task form.

Reclassification Task form

Field	Description
Configuration item	The CI that must be reclassified.
Short description	Short description noting that CI reclassification was not allowed.
Description	Description noting the current class of the CI and the class that the CI must be changed to.
Internal payload	Payload used in the identification process.

What to do next

After examining the task details, you can locate the CI that is noted in the task **Description** and manually reclassify it. For details, see Reclassify a CI.

IRE support for non-CMDB tables

Apply Identification and Reconciliation Engine (IRE) processes to supported non-CMDB tables to ensure data integrity and health of those tables.

Starting with the Zurich release, IRE supports some non-CMDB tables. You can use all IRE features with some non-CMDB tables after creating

identification rules (CI identifiers and identifier entries) for those tables. Non-CMDB tables supported for IRE features include:

- In an application-specific scope: All non-CMDB tables
- In the global scope: Only non-CMDB tables that are preset in the base system. In the Zurich release for example, the Location [cmn_location], Department [cmn_department], Cost Center [cmn_cost_center], Building [cmn_building], User [sys_user], and Group [sys_user_group] non-CMDB tables are supported.

You can't use the CI Class Manager to manage any IRE-related rules for non-CMDB tables. Instead, you must work directly with the respective tables in list views to create and manage those rules as described in the following procedures:

- Create an identification rule for a non-CMDB table
- Create a reconciliation rule for a non-CMDB table
- Create an IRE data source rule for non-CMDB tables
- Create a data refresh rule for a non-CMDB table
- Create an identification inclusion rule for a non-CMDB table
- Simulate payload execution using identification simulation
- Use partial payloads
- Review and remediate de-duplication tasks

You can use the following store apps with supported non-CMDB tables:

- CMDB 360 in CMDB Workspace
- IntegrationHub ETL

IRE processes are applied to supported non-CMDB tables with the following differences:

- IRE doesn't populate the discovery_source, last_discovered, and the first_discovered attributes if those attributes don't exist in the non-CMDB table.

- IRE uses the non-CMDB table's class name as sys_class_name if the table doesn't include a sys_class_name attribute.
- IRE payloads don't support relationships with non-CMDB tables.

  Note: Although IRE-related user interface and accompanying documentation might reference CMDB and CMDB elements, most of those references also apply to any supported non-CMDB tables.

- Create an identification rule for a non-CMDB table

  To use Identification and Reconciliation Engine (IRE) features with supported non-CMDB tables, you must first create identification rules that uniquely identify the table records. Each non-CMDB table can be associated with a single identification rule.

- Create a reconciliation rule for a non-CMDB table

  Create a static or a dynamic CI reconciliation rule for a non-CMDB table.

- Create a data refresh rule for a non-CMDB table

  To apply Identification and Reconciliation Engine (IRE) features to supported non-CMDB tables, create data refresh rules for those tables. Data refresh rules are used to determine if a record is stale for a specific data source. Such records can then be updated by a lower-priority authorized data source.

- Create an identification inclusion rule for a non-CMDB table

  Narrow the scope of records that are included in the identification process of non-CMDB records by creating an identification inclusion rule.

- Create an IRE data source rule for non-CMDB tables

  When using Identification and Reconciliation Engine (IRE), you can prevent a specific data source from inserting new records for a specific non-CMDB table. Create IRE data source rules for data sources that you don't trust in creating records but continue to trust in updating those records that exist.

Create an identification rule for a non-CMDB table

To use Identification and Reconciliation Engine (IRE) features with supported non-CMDB tables, you must first create identification rules that uniquely identify the table records. Each non-CMDB table can be associated with a single identification rule.

Before you begin

Role required: itil has read access, itil_admin (on top of itil) has full access

About this task

Each identification rule consists of a single identifier for the table, one or more identifier entries, and one or more related entries.

Review the following topics before creating identification rules:

- Identification rules
- General guidelines for using CMDB Identification

When creating identifier entries, you can configure the **Search on table** and **Criterion attributes** fields on the Identifier Entry form to implement one of the following options:

Regular identifier entry

Lets you select attributes from the associated identifier table.

Lookup identifier entry

Lets you select attributes from any related table (Lookup table), other than the currently selected table.

Hybrid identifier entry

Lets you select attributes from both the currently main selected table, and from another table (Lookup table).

For non-CMDB tables, only independent identification rules are supported.

Procedure

1. Navigate to **All > Identification/Reconciliation > CI Identifiers**.
2. In the Identifiers list view, click **New**.
3. Fill out the Identifier form.

Field	Description
Name	Name of CI identifier.
Applies to	Supported non-CMDB table.
Independent	Must be checked to indicate that the identifier can identify a record independently of other records.

4. Click **Submit**.
5. In the Identifiers list view, locate and open the identifier that you just created.
6. On the Identifier form, select the **Identifier Entries** tab and then click **New**.
7. Fill out the Identifier Entry form.

Field	Description
Identifier	Preset with the name of the table of the associated identifier.
Search on table	Preset with the label of the table of the associated identifier. To create: • A regular identifier entry: Set to the identifier table and

Field	Description

	select **Criterion attributes** from that same table.
	• A lookup identifier entry: Set to another table (lookup table) and select **Criterion attributes** from that lookup table.
	• A Hybrid identifier entry: Set to another table (lookup table) and then do the following steps.
	  • Select **Criterion attributes** from the lookup table.
	  • Add **Hybrid Entry CI Criterion Attributes** from the current table using background scripts, after saving the rule. For more details, see the 'What to do next' section at the end of this task.
	A lookup table should have a reference to the associated identifier table.

Criterion attributes	Set of attributes that uniquely identify the record. Attributes can belong to the current class, or to a parent class.

Field	Description
	Note: It's possible to add reference fields as a criterion attribute. However, such fields might not always be effective: • Reference fields store sys_ids that point to a record in another table, and thus is considered a weak criterion attribute (in terms of uniqueness) for the current table. • The system detects and then replaces invalid values in a reference field with 'Unknown'. For example, an invalid Model ID value is replaced with the value 'Unknown'. Also, if several CIs end up having that same reference field set to 'Unknown', then these CIs become duplicate CIs.
Priority	Priority of applying the identifier entry. Rules with lower priority numbers are given higher priority. Identifier entries of identical priorities are applied randomly. You can keep gaps between the priority numbers, so you can assign the unused priority numbers to new entries without

Field	Description
	modifying the existing priority order.
Active	Specifies whether the identifier entry is active. At least one identifier entry in an identification rule must be active for the rule to apply.
Enforce exact count match	For lookup identification, match a record only on exact lookup records count match. When enforced, all lookup items for a record in the payload must have matching records in the lookup table that reference the same record: a. Only matches records that have all the lookup items from the input payload referencing the record in the table. b. If there are multiple matches, selects the oldest created record as the final match. When not enforced, one lookup item for a record in the payload matching a record in the lookup table, is sufficient to consider a match: a. Matches any record that has at least one of the lookup items from the input payload referencing the record in the table.

Field	Description
	b. If there are multiple matches, selects the records with the max number of lookup items from the input payload referencing the record in the table. c. If there are still multiple matches, selects the oldest created record as the final match.
Allow null attribute	When selected, then if at least one criterion attribute isn't null, attempt matching with an identifier entry even if there are criterion attributes that are null. Otherwise, all criterion attributes must have values to attempt matching with an identifier entry.
Allow fallback to parent's rules	Allows the identification rules of the record's parent table to be used if a match isn't found for this identification rule. Applies only for dependent identification rules.
Optional condition	A filter to narrow the set of records that will be searched for a matching record. Available only if the glide.identification_engine.enable_identifier_optional_condition system property is set to **true** (**false** by default). In the base system, identifier entries of

Field	Description
	various classes are pre-configured with advanced options conditions. All these pre-configured conditions in regular identifier entries will automatically apply when you set this property to **true**. Therefore, to prevent unexpected behavior, review those predefined conditions in regular identifier entries before setting this property to **true**. For more details about this property, see Properties.

Note: If criterion attributes have only two attributes and sys_class_name is one of them (for example [name, sys_class_name], [ip_address, sys_class_name]), then the other attribute can't be NULL, even if **Allow null attribute** is enabled. This restriction is due to sys_class_name being considered a special system matching attribute.

8. Click **Submit**.
9. On the Identifier form, select the **Related Entries** tab and then click **New**.
10. Fill out the Related Entry form.

Related Entry form

Field	Description
Identifier	Preset with the identifier that this related entry is associated with.
Active	Check box that specifies that the related entry is active.

Field	Description
Related table	A related table (lookup table) that references the record that is being matched.
Referenced field	A referenced field in **Related table** with a reference to the associated identifier table.
Criterion attributes	The set of attributes to uniquely identify the related item. Attributes can belong to the current class, or to a parent class.

Field	Description
	Note: It's possible to add reference fields as a criterion attribute. However, such fields might not always be effective: • Reference fields store sys_ids that point to a record in another table, and thus is considered a weak criterion attribute (in terms of uniqueness) for the current table. • The system detects and then replaces invalid values in a reference field with 'Unknown'. For example, an invalid Model ID value is replaced with the value 'Unknown'. Also, if several CIs end up having that same reference field set to 'Unknown', then these CIs become duplicate CIs. Click the lock icon to view, add, or remove attributes from the identification rule.
Allow null attribute	If at least one criterion attribute in the related table isn't null, allow to attempt matching with an identifier entry even if there are criterion attributes which are null.

Field	Description
Priority	Priority of the related entry for the specified **Related table**. Rules with lower priority numbers are given higher priority while matching a related item for a specific related table. Related entries for the specified related table with identical priorities are applied randomly. You can keep gaps between the priority numbers, so you can assign the unused priority numbers to new entries without modifying the existing priority order.
Optional condition	Filter conditions to narrow the set of records that will be searched for a matching related item.

11. Click **Submit**.

What to do next

To add criterion attributes to a **Hybrid Entry CI Criterion Attributes** field in a hybrid identifier entry, instead of using the Identifier Entry form, you must use background scripts. After saving the identification rule, navigate to **System Definitions > Scripts - Background**, and then enter a script that adds the attributes and click **Run script**.

Sample script:

```
var gr = new GlideRecord('cmdb_identifier_entry');
// get the identifier entry you want to update
gr.get('<identifier_entry_sys_id>');
// set the attributes you want in the hybrid rule in a co
mma separated list
// for example: 'name,serial_number'
gr. hybrid_entry_ci_criterion_attributes='<column_name_1>
```

```
,<column_name_2>,<etc.>';
gr.update();
```

This process requires the admin role.

Create a reconciliation rule for a non-CMDB table

Create a static or a dynamic CI reconciliation rule for a non-CMDB table.

For information about static reconciliation rules, dynamic reconciliation rules, and other principals related to reconciliation rules, see Reconciliation rules.

If both, static and dynamic reconciliation rules exist for the same record attribute, the dynamic rule has precedence.

> **Note:** You can't create a reconciliation rule for system fields or for Identification and Reconciliation Engine (IRE) specific fields such as the Discovery source (discovery_source) field. Also, reconciliation rules can't be dot-walked using reference fields.

Create a static reconciliation rule for a non-CMDB table

A static reconciliation rule specifies class attributes that data sources are authorized to update, and prevents unauthorized data sources from overwriting the attributes' values. A static reconciliation rule also specifies the prioritization among multiple data sources. Without static reconciliation rules, data sources can overwrite each other's updates to attribute values.

Before you begin

Role required: itil has read access, itil_admin (on top of itil) has full access.

About this task

Static reconciliation rules are used in conjunction with data refresh rules to determine reconciliation steps for a record. These rules determine if, when, and by which data source a record can be updated. If multiple data sources are authorized to update the same attributes, assign a priority to each of these data sources to prevent them from overwriting each other's updates.

After an authorized data source updates an attribute, subsequent updates are accepted only from the same data source or from a data source with a higher priority. Updates from a data source with a lower priority are rejected, unless these two conditions are met:

- The lower priority source is the first source updating the record.
- The record became stale based on data refresh rules for the class.

Precedence order of static reconciliation rules:

- Rule configured for a specific attribute, has precedence over rule set with **Apply to all attributes** (regardless of priority value).
- Between two rules for the same attribute or between two rules set with **Apply to all attributes**, the rule that is specific directly for the class has precedence over the derived rule.
- Between two rules for the same attribute or between two rules set with **Apply to all attributes** at the same class level, precedence is determined by rule priorities.

Information about the last discovery source that updates each attribute is stored in the Data Source History [cmdb_datasource_last_update] table, but only after enabling the reconciliation rule. Therefore, there might be unexpected updates after you enable the rule until the highest priority data source has updated the CI.

Static reconciliation rules affect reconciliation of stale attributes. During reconciliation, the information in the Data Source History table is considered along with the data refresh rules for the CI's class, to determine if a CI attribute is stale. A CI attribute is determined to be stale if it was not updated by the latest discovery source to update the CI, within a time period. The time period is specified by the Effective Duration time in the data refresh rule for the class for the discovery source. In this case, if another authorized discovery source, with a lower priority attempts to update the stale CI attribute, the update is allowed.

If there is a dynamic reconciliation rule for the same record attribute as in a static reconciliation rule, the dynamic rule takes precedence.

Procedure

1. Navigate to **All > Configuration > Identification/Reconciliation > Reconciliation Definitions**.
2. In the Reconciliation Definitions list view, click **New**.
3. Fill out the Reconciliation Definition form.

Field	Description
Data source	The data source that you are configuring this rule for.
Priority	Priority of **Data source** within other data sources for the specified attributes. Smaller numbers designate higher priority. Data sources without a reconciliation rule are assigned the lowest priority.
Applies to	Authorizes the specified data source to update all attributes of the specified non-CMDB table. **Note:** This setting will be overridden by any setting that applies to a specific attribute. In which case, instead of using this option, you can directly include all attributes for **Attributes**.
Filter condition	Conditions that records must meet for the rule to be applicable. For example, to apply this rule only to records that are associated with the Finance department, select

Field	Description
	this condition: **[Department] [is] [Finance]**. **Note:** The glide.identification_engine.enable_reconciliation_filter_before_update system property determines when filter conditions are applied. By default, those filter conditions are applied after attribute values have changed during payload processing. Set this property to **true** so that Identification and Reconciliation Engine (IRE) applies the filter conditions before attribute values change.
Attributes	Attributes from the current or from a parent class, that the specified data source is authorized to update. Available only if **Apply to all attributes** is not selected.
Update with null	Attributes that the specified data source can update with a null value. By default, authorized data sources cannot overwrite a non-null value with a null value. Attributes in this list, which are not in the **Attributes** list, are not included with the attributes that

Field	Description
	the data source can update with a null value.

4. Click **Submit**.

Create a dynamic reconciliation rule for a non-CMDB table

A dynamic reconciliation rule for non-CMDB table uses CMDB 360 data to choose a value such as the largest value that is reported, for updating a record.

Before you begin

CMDB 360/Multisource CMDB must be enabled.

Role required: itil has read access, itil_admin (on top of itil) has full access

About this task

If the same CI attribute has both, a static reconciliation rule and a dynamic reconciliation rule, the dynamic reconciliation rule has precedence.

A dynamic reconciliation rule supports several rule types, such as largest reported value and most reported value. When applying a dynamic reconciliation rule, IRE processes the current payload and then examines the CMDB 360 data store to select a value with which to update the CMDB. Depending on the dynamic reconciliation rule type, selecting the appropriate value might not be immediately conclusive. For example, there might not be a single value that is most reported, or for some values, the last discovered timestamp isn't reported. Therefore, when necessary, IRE falls back to examining additional details such as last reported, last discovered, and last updated values to select the most appropriate value.

> **Note:** You can't add a dynamic reconciliation rule when creating a new child class in the CI Class Manager. You must first save the new child class and then add the dynamic reconciliation rule.

Procedure

1. Click **All**.
2. In the Filter navigator, enter `cmdb_dynamic_reconciliation_definition.list` to open the Dynamic Reconciliation Definitions table.
3. In the Dynamic Reconciliation Definitions list view, click **New**.
4. Fill out the Dynamic Reconciliation Definition form.

Field	Description
Name	
Attributes	Attributes for which to apply the rule. Attributes that the specified rule type can't be applied to and attributes for which a dynamic reconciliation rule already exists for, don't appear.
Filter condition	Conditions that CIs must meet for the rule to be applicable. For example, to apply a rule only to CIs that are associated with the Finance department, select this condition: **[Department] [is] [Finance]**.
Applies to	Non-CMDB table that this rule applies to.
Dynamic Rule Type	Rule type which is based on CMDB 360 data.

5. Click **Submit**.

Create a data refresh rule for a non-CMDB table

To apply Identification and Reconciliation Engine (IRE) features to supported non-CMDB tables, create data refresh rules for those tables. Data refresh rules are used to determine if a record is stale for a specific data source. Such records can then be updated by a lower-priority authorized data source.

Before you begin

Role required: itil has read access, itil_admin (on top of itil) has full access

About this task

Data refresh rules have no impact when dynamic reconciliation rules are in effect.

Data refresh rules are used in conjunction with static reconciliation rules to determine reconciliation steps for a record. These rules determine if, when, and by which data source a record can be updated.

Procedure

1. Click **All**.
2. In the Filter navigator, enter `cmdb_datasource_staleness.list` to open the Data Source Staleness Definitions table.
3. In the Data Source Staleness Definitions list view, click **New**.
4. Fill out the Data Source Staleness Definitions form.

Field	Description
Applies to	Non-CMDB class that this rule applies to.
Data source	Data source for which record staleness is evaluated.
Effective Duration	The time period that is used for the staleness evaluation.

Field	Description
	If the fields specified in the static reconciliation rule for the record's class were not updated by the specified data source within the specified time period — the record is determined to be stale for that data source. If you enter a value with a prefix that is valid and a suffix that is not, such as `15 x` — the valid portion of the value is used ('15'). If the entire value is invalid — the default value of 0 is used.
Active	Activates the rule.

5. Click **Submit**.

Create an identification inclusion rule for a non-CMDB table

Narrow the scope of records that are included in the identification process of non-CMDB records by creating an identification inclusion rule.

Before you begin

Role required: itil has read access, itil_admin (on top of itil) has full access.

About this task

During duplication detection of independent CIs, the Identification and Reconciliation Engine (IRE) processes only the records that satisfy the identification inclusion rules. For example, you can set a filter to include only records whose state is operational. When no identification inclusion rules exist, all records are included in the identification process.

In the base system, there are no predefined identification inclusion rules. Identification inclusion rules are defined at the class level.

Note: Identification inclusion rules impact any script that calls IRE, therefore create them carefully. Identification inclusion rules can prevent the identification of certain types of records, affecting some features of Discovery and Service Mapping.

Procedure

1. Navigate to **All > Configuration > Identification/Reconciliation > Identification Inclusion Rules**.
2. In the Identification Inclusion Rules list view, click **New**.
3. Fill out the Identification Inclusion Rules form.

Field	Description
Applies to	Non-CMDB table that this rule applies to.
Inclusion condition	Criteria that non-CMDB records must meet to be included in the identification process.

4. Click **Save**.

Create an IRE data source rule for non-CMDB tables

When using Identification and Reconciliation Engine (IRE), you can prevent a specific data source from inserting new records for a specific non-CMDB table. Create IRE data source rules for data sources that you don't trust in creating records but continue to trust in updating those records that exist.

Before you begin

Role required: itil_admin

About this task

IRE data source rules have no impact when dynamic reconciliation rules are in effect.

- Child classes derive IRE data source rules from parent classes like identification rules do.
- IRE data source rules that are specified for a child class, override any IRE data source rules derived from a parent class.

When IRE processes an insert operation that is prohibited by an IRE data source rule, the insert operation fails. This failure happens when the data source and record class in the insert operation and in an IRE data source rule, match. When CreateOrUpdateCIEnhanced() is used, IRE stores the failed payload in the CMDB IRE Partial Payloads [cmdb_ire_partial_payloads] table for future potential use.

> **Note:** When an insert operation is not allowed by the IRE data source rule, then when using createOrUpdateCI(), the entire IRE payload fails since createOrUpdateCI() doesn't allow partial commits.

If later, a permitted data source attempts to insert that same record, then IRE inserts the record after merging it with the matching record from the partial payloads. IRE then deletes the partial payload from the CMDB IRE Partial Payloads [cmdb_ire_partial_payloads] table, and allows future updates by the data source specified in the rule.

IRE data source rules do not apply to lookup and related items, and only a single rule can be active for any class/data source pair.

Procedure

1. Navigate to **All > Configuration > Identification/Reconciliation > IRE Data Source Rules**.
2. In the IRE Data Source Rules list view, click **New** and fill out the IRE Data Source Rule form.

Field	Description
Data source	Data source that is not allowed to create CIs of the specified class.
Active	Activates the IRE data source rule.

Field	Description
Applies to	The class (and child classes) that the specified data source is not allowed to create records for.
Insert Not Allowed	Disables the specified data source from inserting new records of the specified class, to the non-CMDB table.

3. Click **Submit**.

Result

If a payload item with an insert request, and in which the data source and the record class match the data source and the record class specified in the IRE data source rule:

1. The insert operation fails and IRE logs the following message:

   INSERT_NOT_ALLOWED_FOR_SOURCE Insert into [xyz] is blocked for data source [xyz] by IRE data source rule.

2. If using CreateOrUpdateCIEnhanced(), then IRE stores the payload item as a partial payload in the CMDB IRE Partial Payloads [cmdb_ire_partial_payloads] table.

If later, a permitted data source successfully inserts a record that matches the record from a partial payload item:

1. The current record is merged with the matching record from the partial payload, applying static reconciliation rules as needed.
2. The respective partial payload in the CMDB IRE Partial Payloads [cmdb_ire_partial_payloads] table is deleted.
3. Later payloads in which the non-permitted data source updates the respective record, run successfully.
4. IRE allows the data source, that was previously prohibited from inserting the record, to update that same record which now exists in the non-CMDB table.

CMDB IRE reference

Reference topics provide property settings, domain separation, and other reference content for the CMDB Identification and Reconciliation Engine (IRE).

Reference topics

Domain separation

How domain separation is supported in IRE to let you separate data, processes, and administrative tasks by domains in your organization.

Properties

Properties that you can adjust to manage different aspects of how IRE functions.

IRE error messages

Errors and messages that the Identification and Reconciliation Engine (IRE) generates.

Components installed with IRE

Details about tables associated with IRE.

Domain separation

Domain separation is supported in the CMDB Identification and Reconciliation feature. Domain separation enables you to separate data, processes, and administrative tasks into logical groupings called domains. You can control several aspects of this separation, including which users can see and access data.

Overview

Domain separation is enforced during the CMDB Identification and Reconciliation (IRE) process. IRE processes are domain aware and domain separation is applied to the Identification and Reconciliation rules.

For more information about domain separation, see domain separation.

How domain separation works in Identification and Reconciliation

Domain separation in the identification engine is enforced when users activate the domain separation plugin. Domain separation for IRE has two modes of operation in domain separated instances:

- Strict mode (enabled by default): In this mode, identification processes only those CIs in which the domain ID is identical to the domain of the currently logged in user. If duplicate CIs exist across domains (including parent and child domains), then those CIs aren't considered duplicate CIs because their domain IDs don't match.
- Platform domain separation mode (disabled by default): In this mode, IRE follows the platform domain separation behavior. So during identification, parent domains can access all CIs within their child domains or any of the domains it has visibility into. For more information, see Visibility domains and Contains domains.

  Platform domain separation mode is intended to be used by advanced users for very specific or advanced use cases.

  Note:

  Platform domain separation mode introduces some risks that are greater on upgraded instances and much lesser on zBooted instances.

  Depending on how IRE processes are configured on a domain separated instance, setting IRE to use platform domain separation mode might result in unexpected and undesirable behavior if not used carefully. One of the risks is if enabling platform domain separation mode is followed by running IRE processes from a different domain than the one on which IRE processes were previously run. In this situation, CIs that were previously identified as unique, might get identified as duplicate CIs and might cause some applications to start failing.

  If any application is already using IRE effectively in domain separated environment, then there's no advantage in switching to platform domain separation mode that might create some risk.

Use the glide.identification_engine.platform_domain_separation_enabled system property to switch between those two modes for IRE domain separation. By default, this property is set to **false**.

Platform domain separation mode

Set the system property glide.identification_engine.platform_domain_separation_enabled to **true** to enable the platform domain separation mode for IRE processing. With the platform domain separation mode, parent domains can access all of their child domains during IRE processing. For example, IRE can detect a matching CI in a child domain and then update that CI instead of creating a new one.

In the platform domain separation mode for IRE:

- IRE run from a parent domain can access CIs contained within their domain, child domains that are lower in the domain hierarchy, and global domain.
- IRE run from the global domain can access all CIs.
- Visibility domains and Contains domains are supported.

> **Note:** When platform domain separation mode is enabled, there might be a sudden increase in IRE detection of duplicate CIs.

Domain separation during the Identification Process

Domain separation during the Identification process is enforced as follows:

- Regardless of the setting of the glide.identification_engine.platform_domain_separation_enabled system property:
  - Domain IDs don't need to be explicitly sent in the input payload of the identification engine APIs. Internally, the identification engine causes the current domain ID of the user to call the identification engine APIs.
  - During matching, if no records are found and a CI is inserted, the CI domain ID is the same as the domain ID of the logged-in user's domain. When updating a CI, the CI domain ID doesn't change.

  - During matching, if duplicates are found, De-Duplication tasks created in the [reconcile_duplicate_task] table have the same domain ID as of the duplicate CIs.
  - During matching, if reclassification of the CI isn't allowed, reclassification tasks are created in the [reclassification_task] table, with the same domain ID as the CI for which reclassification is needed.
- When the system property glide.identification_engine.platform_domain_separation_enabled is set to **false**:
  - Only CIs that have the same domain ID as the currently logged-in user's domain are used during matching.
  - Duplicate CIs that exist across domains (including parent and child domains) aren't considered as duplicate CIs by IRE.
- When the system property glide.identification_engine.platform_domain_separation_enabled is set to **true**:
  - Duplicate CIs that exist across domains (such as parent and child domains) are considered as duplicate CIs by IRE.
  - CIs from the logged in user domain and child domains are used during matching.

Domain separation and Identification Rules

The identification rules and identification inclusion rules used during the identification process are always defined at the global level. For example, the following tables don't have a sys_domain field:

- Identifier (cmdb_identifier)
- Identifier Entries (cmdb_identifier_entry)
- Related Entries (cmdb_related_entry)
- Identification Inclusion Rules (cmdb_ie_active_config)

Domain separation and Reconciliation Rules

The reconciliation definition rules that are used during the reconciliation process can be defined for different domains. For example, the following tables do have sys_domain, sys_overrides, sys_domain_path fields:

- Reconciliation Definition (cmdb_reconciliation_definition)
- Datasource Precedence (cmdb_datasource_precedence)
- Data Source Staleness Definitions (cmdb_datasource_staleness)

Properties

Use the Identification and Reconciliation properties to configure the identification and reconciliation engine (IRE).

These properties are available for Identification and Reconciliation. To view and edit these properties, the admin role is required.

> **Note:** To open the System Properties [sys_properties] table, enter `sys_properties.list` in the navigation filter.

Properties for Identification and Reconciliation

Property	Description
Enforce the requirement that required attributes cannot be null during identification and reconciliation. glide.required.attribute.enabled	• Type: true | false • Default value: true • Location: **Configuration > CMDB Properties > Identification/ Reconciliation Properties**
Allow class upgrade during IRE identification and reconciliation. glide.class.upgrade.enabled	• Type: true | false • Default value: true • Location: **Configuration > CMDB Properties > Identification/ Reconciliation Properties**

Property	Description
	• Learn more: Configure CI reclassification during IRE processing. When **false**, IRE rejects a payload (or a payload item in Enhanced IRE) with the respective reclassification update, and creates a reclassification task.
Allow class downgrades during IRE identification and reconciliation. glide.class.downgrade.enabled	• Type: true | false • Default value: true • Location: **Configuration > CMDB Properties > Identification/ Reconciliation Properties** • Learn more: Configure CI reclassification during IRE processing. When **false**, IRE rejects a payload (or a payload item in Enhanced IRE) with the respective reclassification update, and creates a reclassification task.
Allow class switching during IRE identification and reconciliation. glide.class.switch.enabled	• Type: true | false • Default value: true • Location: **Configuration > CMDB Properties > Identification/ Reconciliation Properties**

Property	Description
	• Learn more: Configure CI reclassification during IRE processing. When **false**, IRE rejects a payload (or a payload item in Enhanced IRE) with the respective reclassification update, and creates a reclassification task.
glide.identification_engine.update_without_upgrade_enabled	Enable IRE to process CI updates with upgrade reclassification updates. This property takes precedence over the glide.class.upgrade.enabled property. • Type: true | false • Default value: false • Location: Add to System Properties [sys_properties] table. • Learn more: Configure CI reclassification during IRE processing. Depending on the property setting, IRE processes or skips the upgrade update: • **true**: IRE processes the CI updates but doesn't process the CI upgrade reclassification update.

Property	Description
	• **false**: IRE processes the CI updates including the CI upgrade reclassification update.
glide.identification_engine.update_without_downgrade_enabled	Enable IRE to process CI updates with downgrade reclassification updates. This property takes precedence over the glide.class.downgrade.enabled property. • Type: true | false • Default value: false • Location: Add to System Properties [sys_properties] table. • Learn more: Configure CI reclassification during IRE processing. Depending on the property setting, IRE processes or skips the downgrade update: • **true**: IRE processes the CI updates, but doesn't process the CI downgrade reclassification update. • **false**: IRE processes the CI updates including the CI downgrade reclassification update.

Property	Description
glide.identification_engine.update_without_switch_enabled	Enable IRE to process CI updates with switch reclassification updates. This property takes precedence over the glide.class.switch.enabled property. • Type: true | false • Default value: false • Location: Add to System Properties [sys_properties] table. • Learn more: Configure CI reclassification during IRE processing. Depending on the property setting, IRE processes or skips the switch update: • **true**: IRE processes the CI updates, but doesn't process the CI switch reclassification update. • **false**: IRE processes the CI updates including the CI switch reclassification update.
glide.identification_engine.reclassification_restriction_rules_enabled	Globally enable or disable the application of active reclassification restriction rules. • Type: true | false • Default value: true • Location: Add to System Properties [sys_properties] table.

Property	Description
	• Learn more: Configure CI reclassification during IRE processing.
Allow the update of an empty field by a lower priority data source. glide.reconciliation.override.null	• Type: true | false • Default value: true • Location: **Configuration > CMDB Properties > Identification/ Reconciliation Properties**
Controls how identification processes a small set of duplicate CIs. glide.identification_engine.skip_duplicates	• Type: true | false • Default value: true • Other values: **true** If the number of duplicate CIs is less than the threshold specified by glide.identification_engine.skip_duplicates.threshold, then the oldest of the duplicate CIs is picked as a match and gets updated. That oldest CI is also designated as the main CI for that set of duplicate CIs. For the rest of the duplicate CIs, the duplicate_of field is set as a reference to the main CI.

Property	Description
	false Matching a CI fails, and an error is logged. • Location: **Configuration > CMDB Properties > Identification/ Reconciliation Properties**
Maximum number of CIs that can be in a set of duplicate CIs to allow identification to process the duplicate CIs according to the setting of glide.identification_engine.skip_duplicates. glide.identification_engine.skip_duplicates.threshold	If the number of duplicate CIs exceeds the threshold, then identification processes the duplicate CIs as if glide.identification_engine.skip_duplicates is set to false. • Type: Integer • Default value: 5 • Location: **Configuration > CMDB Properties > Identification/ Reconciliation Properties**
Maximum number of log runs that can be displayed when navigating to **Configuration > Identification Logs**. glide.identification_logs.max_run_ids	• Type: integer • Default value: 1000 • Location: **Configuration > CMDB Properties > Identification/ Reconciliation Properties**
glide.cache.size.service_cache	Maximum cache size (in MB) that is used by the identification engine for inbound and outbound relations. When the limit is reached, the least recently

Property	Description
	used cached data is discarded, releasing space for new data. **Note:** You cannot disable the service cache. • Type: Integer • Default value: 20 • Location: Add to System Properties [sys_properties] table.
glide.identification_engine.granular_insert_locking	Determines whether to use multiple granular insert locks or single global insert lock. Set to false if there are performance issues associated with the usage of multiple granular insert locks. • Type: true | false • Default value: true • Location: Add to System Properties [sys_properties] table.
glide.identification_engine.batch_update_last_discovered	Controls batch update of last_discovered field in CIs that are being processed by the identification engine. Set to false if there are business rules that apply to the last_discovered field, and you want to trigger these rules

Property	Description
	when calling an Identification and Reconciliation API. • Type: true | false • Default value: true • Location: Add to System Properties [sys_properties] table.
glide.identification_engine.related_items_local_cache_count	For optimization, a custom number of locally cached query result entries of related/lookup items. • Type: integer • Default value: 15000 • Location: Add to System Properties [sys_properties] table. **Note:** If there is a memory issue due to optimization related to using local cache, set the glide.identification_engine.related_items_local_cache_count and the glide.identification_engine.dependent_items_local_cache_count properties to 0.
glide.identification_engine.dependent_items_local_cache_count	For optimization, a custom number of locally cached query result entries of dependent CIs. • Type: integer • Default value: 10000

Property	Description
	• Location: Add to System Properties [sys_properties] table. **Note:** If there is a memory issue due to optimization related to using local cache, set the glide.identification_engine.related_items_local_cache_count and the glide.identification_engine.dependent_items_local_cache_count properties to 0.
glide.identification_engine.independent_items_local_cache_count	For optimization, a custom number of locally cached query result entries of independent CIs. • Type: integer • Default value: 100000 • Location: Add to System Properties [sys_properties] table. Setting the value to 0 avoids using local cache for independent CIs which might affect performance.
glide.cmdb.logger.source.identification_engine	Enable and configure what type of details the system logs when using IRE outside the scope of identification simulation. For example, when using an API, ECC queue or scheduled jobs. • Type: string

Property	Description
	• Values: info, warn, error, debug, or debugVerbose • Location: Add to System Properties [sys_properties] table. **Note:** Depending on the setting, the system can generate large amounts of data that might affect overall system performance. Set the value with caution, and limit the level of details and use time to the minimum necessary for testing or debugging. For more troubleshooting information, see the How to capture IRE [identification and reconciliation engine] debug logs [KB0750382] knowledge base article.
glide.identification_engine.partial_payload_items_max_size	Maximum number of items allowed when creating a partial payload. When that limit is reached, the partial payload is split. For example, when IRE creates a partial payload, items and associated relations and references, are all merged in one partial payload. This merge could result in a large partial payload.

Property	Description
	Adjusting this property can help with performance issues related to IRE processing of partial items. • Type: integer • Default: 1000 • Learn more: Identification and Reconciliation engine (IRE) • Location: Add to System Properties [sys_properties] table
glide.identification_engine.partial_items_process_limit	Maximum number of partial items to be fetched in a single IRE call. After reaching this limit, IRE fetches only partial items corresponding to complete items in the input payload. Adjusting the value can help with performance issues related to IRE processing of partial items. • Type: integer • Default: 2000 • Location: Add to System Properties [sys_properties] table.
glide.identification_engine.partial_items_process_absolute_limit	Absolute limit of the number of partial items for IRE to fetch, after which, IRE stops fetching partial payloads from the CMDB IRE Partial Payloads [cmdb_ire_partial_payloads] table. Adjusting the value can help with

Property	Description
	performance issues related to IRE processing of partial items. • Type: integer • Default: 5000 • Location: Add to System Properties [sys_properties] table.
glide.identification_engine.skip_updating_source_last_discovered_if_older	Determines how IRE updates the last_discovered and the discovery_source attributes in the CMDB. • true: If last_discovered is provided in the payload and it is older than the last_discovered of the CI in the CMDB, IRE does not use the payload values to update the last_discovered and the discovery_source attributes in the CMDB. • false: Even if the last_discovered provided in the payload is older than the last_discovered of the CI in the CMDB, IRE uses the payload values to update the last_discovered and the discovery_source attributes in the CMDB. **Note:** Only the attributes mentioned above are affected by this property in an update operation. • Type: true | false

Property	Description
	• Default: true • Learn more: Identification and Reconciliation engine (IRE) • Location: Add to System Properties [sys_properties] table.
glide.identification_engine.ire_message_listener_skip_updating_source_last_discovered_to_now	If Robust Transform Engine (RTE) does not pass the ire.skip_updating_last_scan_to_now custom property on the Robust Import Set Transformer form, IRE uses the value of this property for the skip_updating_source_last_discovered_to_now IRE option. • Type: true | false • Default: false • Learn more: Identification and Reconciliation engine (IRE) • Location: Add to System Properties [sys_properties] table.
glide.identification_engine.skip_updating_last_scan_if_older	Determines how IRE uses the source_recency_timestamp value in a payload to determine whether to update the last_scan attribute in the Source [sys_object_source] table. • true: If source_recency_timestamp is provided in the payload and it is older than the last_scan of the CI in the CMDB, IRE does not

Property	Description
	update the last_scan attribute in the Source [sys_object_source] table. • false: Even if the source_recency_timestamp provided in the payload is older than the last_scan of the CI in the CMDB, IRE uses the payload value to update the last_scan attribute in the Source [sys_object_source] table. You can check the input payload for a CI and the last_scan attribute value in the Source [sys_object_source] table to learn if IRE will update that last_scan value or not. **Note:** Only the attributes mentioned above are affected by this property in an update operation. • Type: true | false • Default: true • Learn more: About mapping data columns to CMDB classes and attributes and Identification and Reconciliation engine (IRE) • Location: Add to System Properties [sys_properties] table.
glide.identification_engine.ire_message_listener_skip_updating_last_scan_to_now	If RTE does not pass the ire.skip_updating_last_scan_to_no

Property	Description
	w custom property on the Robust Import Set Transformer form, IRE uses the value of this property for the ire.skip_updating_last_scan_to_now IRE option. • Type: true | false • Default: false • Learn more: About mapping data columns to CMDB classes and attributes and Identification and Reconciliation engine (IRE) • Location: Add to System Properties [sys_properties] table.
glide.identification_engine.platform_domain_separation_enabled	Toggles domain separation support mode during IRE processing. • false: IRE processes run only within the current domain. Basically disabling parent domains access to child domains during IRE processing. • true: IRE domain separation follows the platform domain separation behavior. Basically, enabling parent domains to look access into all its child domains during IRE processing. • Type: true | false • Default: false • Learn more: Domain separation

Property	Description
	• Location: Add to System Properties [sys_properties] table.
glide.identification_engine.enable_identifier_optional_condition	Enables advanced options for regular identifier entries in identification rules. Those advanced options let you add conditions to narrow the set of records that will be searched for a matching CI. **Note:** This property affects only regular identifier entries (it doesn't affect lookup or hybrid identifier entries). In the base system, identifier entries of various classes are pre-configured with advanced options conditions. All these pre-configured conditions in regular identifier entries will automatically apply when you set this property to **true**. To prevent unexpected behavior, review those predefined conditions in regular identifier entries before setting this property to **true**. In the **Filter** box in the primary navigation, enter `cmdb_identifier_entry.list`. Then, in the Identifier Entry list view, review the 'Optional condition' column.

Property	Description
	• Type: true | false • Default: false • Learn more: Create a CI identification rule • Location: Add to System Properties [sys_properties] table.
glide.identification_engine.enable_reconciliation_filter_before_update	Determines whether filter conditions of a reconciliation rule are applied before a value change during payload processing, or after. • Type: true | false • Default: false • Learn more: Create a static reconciliation rule, Create a dynamic reconciliation rule • Location: Add to System Properties [sys_properties] table.
glide.identification_engine.skip_sys_object_source_matching	Determines whether IRE identification processes have the priority in being used to uniquely identify CIs in a payload, over the use of sys_object_source lookup. When set to true, the system prioritizes sending any payload that contains a criterion attribute to be processed by IRE identification instead of using sys_object_source lookup. This can

Property	Description
	be useful in situation that can potentially generate duplicate CIs. For example, using sources in which identifying attributes are changing while the source native key isn't. However, when source_native_key is the only identifiable attribute (for example, if none of the identification rules can run because attributes and values for rule identifiers aren't present in the payload), then source_native_key is used for identification even when the property is set to **true**. • Type: true | false • Default: false • Learn more: Identification and Reconciliation engine (IRE) • Location: Add to System Properties [sys_properties] table.

IRE error messages

The Identification and Reconciliation Engine (IRE) generates the following errors and messages. Depending on settings, these messages appear in the Identification Logging pane and in the system logs.

For information about lookup-based CI identification and qualifier chains, see Create a CI identification rule.

> **Note:** IRE performs an initial verification of a payload before processing identification rules. If IRE detects any duplicate CIs based on any class identifiers, the payload is rejected and processing stops.

For information about CMDB Identification Payload error: "FAILED TRYING TO EXECUTE ON CONNECTION", see CMDB Identification Payload error - "Insertion failed with error Error during insert of cmdb_ci...", where node logs show "FAILED TRYING TO EXECUTE ON CONNECTION" "Duplicate entry 'XXX' for key 'XXX"' knowledge base article.

Error- IDENTIFICATION_RULE_MISSING

Message	Description and Resolution
Identity Rule Missing for table [xyz]	**Description:** Identification rule is missing for a class. **Resolution:** Ensure that there is an identification rule for table [xyz], and that the rule is active.

MISSING_MATCHING_ATTRIBUTES

Message	Description and Resolution
In payload missing minimum set of input values for criterion (matching) attributes from identify rule for table [xyz]. Add these input values in payload item 'abc'	**Description:** Missing minimum set of values for criterion attributes for an identification rule. **Resolution:** In the payload, add minimum set of values for criterion attributes for CI Identifier for table [xyz]. Open the CI Class Manager, click **Hierarchy** and select the [xyz class.

Message	Description and Resolution
	Check the identification rule and the identifier entries for table [xyz.

Error- NO_CLASS_NAME_FOR_INDEPENDENT_CI

Message	Description and Resolution
Cannot have 'sys_class_name' as a key field in an Independent Identity Rule on 'xyz'	**Description:** The class attribute was added to the CI identifier which is not supported. **Resolution:** Remove the class attribute from CI Identifier for table [xyz].

Error- IDENTIFICATION_RULE_FOR_LOOKUP_MISSING

Message	Description and Resolution
Identity Rule for table [xyz] missing Lookup Rule for class [abc]	**Description:** The payload has a lookup class name, but the corresponding lookup rule is missing. **Resolution:** Add lookup identifier entry with [Search on table] as [abc] for CI Identifier for table [xyz]. For more information about this error message, see

Message	Description and Resolution
	Identification error "Identity Rule for table [cmdb_ci_table_name] missing Lookup Rule for class [table_name]" [KB0786444].

Error- IDENTIFICATION_RULE_FOR_RELATED_ITEM_MISSING

Message	Description and Resolution
Identity Rule for table [xyz] missing Related Rule for class [abc]	**Description:** The payload has a related class name, but the corresponding related rule is missing. **Resolution:** Add related entry with [Related table] as [abc] within CI Identifier for table [xyz].

Error- NO_LOOKUP_RULES_FOR_DEPENDENT_CI

Message	Description and Resolution
Cannot have Lookup Rule for a Dependent Identity Rule on 'xyz'	**Description:** Cannot have Lookup Rule for a Dependent Identity Rule. **Resolution:** Remove lookup identifier entry from dependent CI Identifier for table [xyz].

Error- INVALID_INPUT_DATA

Message	Description and Resolution
Found invalid sys_id in payload. No record with sys_id [xyz] exist in table [abc] or is a duplicate record with [duplicate_of] field set to a main CI	**Description:** The payload has a reference to an invalid sys_id. **Resolution:** Remove the referenced sys_id, or provide a valid sys_id.
In payload no data source exist. You need to provide choice value from choice field [discovery_source] in table [cmdb_ci]	**Description:** In payload no data source exists. **Resolution:** In the payload, provide a valid choice value from choice field [discovery_source] from table [cmdb_ci].
In payload invalid data source [xyz] exist. You need to provide a valid choice value from field [discovery_source] in table [cmdb_ci]	**Description:** The payload contains an invalid data source. **Resolution:** In the payload, provide a valid choice value from choice field [discovery_source] from table [cmdb_ci].

Message	Description and Resolution
No such relationship with name [xyz] exist in table [cmdb_rel_type]. If out-of-box relationship for [xyz] has been removed or renamed, it should be restored	**Description:** The payload is referencing a relationship that does not exist in the [cmdb_rel_type] table. **Resolution:** Verify that the reference to the relationship is accurate. Or, if it is a new relationship, add it to the [cmdb_rel_type] table. Or, If out-of-box relationship for [xyz] has been removed or renamed, restore it.
Payload relations 'xyz' has invalid parent record index: [0]	**Description:** Payload references invalid parent indexes. **Resolution:** Check payload indexes and ensure that they are all valid.
Payload relations 'xyz' has invalid child record index: [0]	**Description:** Payload references invalid child indexes. **Resolution:** Check payload indexes and ensure that they are all valid.

Error- DUPLICATE_RELATIONSHIP_TYPES

Message	Description and Resolution
Duplicate relationship type records exists with name [xyz] in table [cmdb_rel_type] having sys_ids: [abc]	**Description:** There are duplicate records in the [rel_ci_type] table for the relationship. **Resolution:** Remove the duplicate records.

Error- DUPLICATE_PAYLOAD_RECORDS

Message	Description and Resolution
Found duplicate items in the payload (index 0 and 1), using className [xyz] and fields [abc]. Remove duplicate items from payload	**Description:** The payload contains two items whose criterion attributes have identical values. **Resolution:** Remove one of the duplicate items.

Error- LOCK_TIMEOUT

Message	Description and Resolution
Failed to acquire synchronization lock for xyz	**Description:** Failed to acquire the system mutex lock. **Resolution:** Increase the mutex expiration time by adding the system property **glide.identification_engine.mutex_expiration_time** and setting to an integer value that is greater than the default value (15 min).

Error- MULTIPLE_DUPLICATE_RECORDS

Message	Description and Resolution
Found duplicate records in table [xyz] using fields [abc]	**Description:** Found duplicate records in the specified table. **Resolution:** Fix the duplicate records found by the identification engine. Check de-duplication tasks for information about all duplicates.

Error- REQUIRED_ATTRIBUTE_EMPTY

Message	Description and Resolution
Missing mandatory field [xyz] in table [abc]. Add input value for mandatory field in payload	**Description:** A required attribute is missing in the payload. **Resolution:** In the payload, add input value for mandatory field [xyz] in table [abc].

Error- MISSING_DEPENDENCY

Message	Description and Resolution
In payload no relations defined for dependent class [xyz] that matches any containment/hosting rules: [abc]. Add appropriate relations in payload for 'def'	**Description:** No relations defined for the dependent class that matches any of its metadata rules. **Resolution:** In payload add appropriate relations for dependent class [xyz] that matches any containment/ hosting rules: [abc].

Error- METADATA_RULE_MISSING

Message	Description and Resolution
No containment or hosting rules defined for dependent class [xyz]. Add containment/hosting rules for 'abc'	**Description:** There are no containment or hosting rules defined for dependent class. **Resolution:** Add containment or hosting rules for dependent class [xyz].

Error- MULTIPLE_DEPENDENCIES

Message	Description and Resolution
Found multiple dependent relation items [xyz] and [abc] in payload	**Description:** Multiple dependent relation items exist. **Resolution:** Remove one of the multiple dependent relation items [xyz] or [abc].
Multiple paths leading to the same destination: xyz -> abc	**Description:** Multiple paths leading to the same destination.

Message	Description and Resolution
	Resolution: Remove duplicate relationship/ qualifier chains that might exists between xyz -> abc.

Error- ABANDONED

Message	Description and Resolution
Abandoning processing payload item 'xyz', since its depends on payload item 'abc' has errors	**Description:** Dependent payload item has errors, so abandoning processing. **Resolution:** Resolve the error on the dependent payload item ‘abc'.
Can't find matched record with sys_id [xyz] in table [abc]	**Description:** Matched sys_id does not exist in the corresponding table. **Resolution:** Check in table [abc] whether matched record is a valid record based on input payload.
Identification engine API got called recursively, aborting...	**Description:** The Identification engine API was called recursively.

Message	Description and Resolution
	Resolution: Avoid calling the Identification engine API recursively.
Detected error while processing payload from xyz	**Description:** Error occurred during processing payload. **Resolution:** Resolve all errors mentioned in the output payload from xyz.
While processing relations encountered errors in payload item: xyz	**Description:** Payload item has errors. **Resolution:** Resolve errors in payload item 'xyz'.
Error occurred during parsing input json payload: xyz	**Description:** Error occurred during parsing JSON payload. **Resolution:** Ensure that input JSON payload has correct JSON format.

Error- MULTI_MATCH

Message	Description and Resolution
Duplicate dependent records found having relationship [xyz] with same CI (className:[abc], sysId: [def])	**Description:** Found duplicate dependent CIs. **Resolution:** Check de-duplication tasks for information about all duplicates, and then delete duplicate records.
Found multiple relations between payload items: 'xyz' and 'abc'	**Description:** Found multiple relations between payload items. **Resolution:** Check for duplicate relationship chains and qualifier chains that might exist.
Found duplicate records in lookup table [xyz] using fields [abc] and reference field [def]	**Description:** Found duplicate records in lookup table. **Resolution:** Check de-duplication tasks for information about all duplicates, and then delete duplicate records.

Error- QUALIFICATION_LOOP

Message	Description and Resolution
Qualification chain has loop that contains relation 'xyz'	**Description:** Qualification chain has a loop. **Resolution:** Remove the loop from the qualification chain with relation ‘xyz'.

Error- TYPE_CONFLICT_IN_QUALIFICATION

Message	Description and Resolution
Invalid payload, qualification chain has multiple possible paths for payload items: 'xyz' and 'abc'	**Description:** Multiple qualification paths found. **Resolution:** Remove multiple possible qualification paths between items ‘xyz’ and ‘abc'.

Error- RECLASSIFICATION_NOT_ALLOWED

Message	Description and Resolution
CI Reclassification not allowed from class: [xyz] to [abc]	**Description:** CI reclassification not allowed.

Message	Description and Resolution
	Resolution: Check reclassification tasks for information about reclassification, and check if reclassification from class: [xyz] to [abc] is valid.

Error- DUPLICATE_RELATED_PAYLOAD

Message	Description and Resolution
Found duplicate Related items (0 and 1) in the payload index 1 using fields xyz	**Description:** Duplicate Related items present. **Resolution:** Remove one of the duplicate related items present in the payload.

Error- DUPLICATE_LOOKUP_PAYLOAD

Message	Description and Resolution
Found duplicate Lookup items (0 and 1) in the payload index 1 using fields xyz	**Description:** Duplicate lookup items present. **Resolution:** Remove one of the duplicate lookup items present in the payload.

INSERT_NOT_ALLOWED_FOR_SOURCE

Message	Description and Resolution
Insert into [xyz] is blocked for data source [abc] by IRE data source rule	**Description:** An IRE data source rule is configured to prevent data source [abc] from inserting CIs of the [xyz] class. **Resolution:** Delete or update the appropriate IRE data source rule to let data source [abc] insert CIs of the [xyz] class. Or, wait for another permitted data source to create the same CI.

Components installed with IRE

Several types of components are installed with Identification and Reconciliation (included in the com.snc.cmdb plugin), including tables.

Tables installed

Table	Description
Identifier [cmdb_identifier]	Identification rule sets defined for different classes of CIs.
Reconciliation Definition	Static reconciliation rules defined for different classes of CIs at the table and field level.

Table	Description
[cmdb_reconciliation_definition]	
Dynamic Reconciliation Definitions [cmdb_dynamic_reconciliation_definition]	Dynamic reconciliation rules defined for different attributes and classes.
Identifier Entry [cmdb_identifier_entry]	Rule entries with different priorities assigned to each identifier.
Duplicate Audit Result [duplicate_audit_result]	Duplicate audit results corresponding to a specific duplicate task. These results are generated automatically during the identification process and should not be added manually.
Remediate Duplicate Task [reconcile_duplicate_task]	Task to address duplication that is detected during the identification process. Records are generated automatically, and users should not add records manually.
Reclassification Task [reclassification_task]	Reclassification tasks that were generated during the identification process.
Data Source History [cmdb_datasource_last_update]	Information about the last data source that updated each attribute. Used to determine if a data source can update a stale CI.
Data Source Staleness Definition	Effective duration per data source. When effective duration is exceeded, then CMDB Health determines that the information

Table	Description
[cmdb_datasource_staleness]	provided by that data source is stale.
Identification Engine Context [cmdb_ie_context]	Input payload, and data source (cmdb_ci's discovery_source) that will be used as input for a specific identification engine API. Stores information about which specific identification engine API will be called (identifyCI or createOrUpdateCI API). Also stores information about enhanced IRE options used in Identification Simulation. **Note:** Internal table used by identification simulation.
Identification Engine Run [cmdb_ie_run]	Specific cmdb_ie_context record that was used to run against the identification engine. Also details about the output payload returned by APIs, such as start and end time of the run and whether the run was successful. **Note:** Internal table used by identification simulation.
Identification Engine Log [cmdb_ie_log]	Identification engine logs for a specific cmdb_ie_run simulated in the identification simulation. Also details about logs level and order.

Table	Description
	Note: Internal table used by identification simulation.
IRE Data Source Rule [cmdb_ire_data_source_rule]	IRE data source rules.
CMDB IRE Partial Payloads [cmdb_ire_partial_payloads]	Payload items that were determined to be partial, and which might be later matched with an incoming payload. If a partial payload is matched and processed, it is deleted from the CMDB IRE Partial Payloads table. Partial payloads older than 90 days are deleted from the table. For more information about usage of this table in IRE processes, see Identification and Reconciliation engine (IRE).
CMDB IRE Partial Payloads Index [cmdb_ire_partial_payloads_index]	Identifier keys associated with partial items. IRE uses those keys to try to match with identifier keys of incoming payloads. For more information about usage of this table in IRE processes, see Identification and Reconciliation engine (IRE).
CMDB IRE Incomplete Payloads	Incomplete items, stored using JSON format as incomplete

Table	Description
[cmdb_ire_incomplete_payloads]	payloads. Incomplete items are stored for the purpose of logging payloads with irrecoverable errors, and are never processed again. The table is configured for table rotation, with duration of one day and seven table rotations. For more information about usage of this table in IRE processes, see Identification and Reconciliation engine (IRE).
IRE Output Aggregate Stats [cmdb_ire_output_aggregate_stats]	This table is populated when RTE invokes IRE, for example, when processing integrations. Details about data inserted by Import Sets or Robust Transform Engine (RTE) to the CMDB (via IRE). Numbers of items inserted, partial items, and updated items, are stored for each type of CI, per run.
IRE Output Target Items [cmdb_ire_output_target_item]	This table is populated when RTE invokes IRE, for example, when processing integrations. Details about data inserted by Import Sets or Robust Transform Engine (RTE) to the CMDB (via IRE). Target class and the sys_id are stored per ImportSet row id, within a run.

Table	Description
	For this table to populate, RTE must pass the ire_output_detailed_stats property.
Reclassification Restrictions [cmdb_ire_reclassification_restriction]	Reclassification restriction rules. These rules prevent switch and downgrade reclassification updates for specific source and target classes. For more information, see Configure CI reclassification during IRE processing.

User roles installed

Role	Description
cmdb_payload_admin	Automatically assigned to users with the cmdb_admin role for internal use only.

Related reference

- Properties